Django for Dummies

Learn Django by taking apart a real app: the Daily Schedule planner

Theory first, with diagrams and worked examples. Then every file in the project, line by line: why it is written that way, what would happen otherwise, and simpler or alternative ways (such as class-based views).

Contents

1. How to use this guide 8

Conventions used in the boxes . . . 8

How to teach with it . . . 8

The five ideas that repeat everywhere . . . 8

2. Languages used, and what to learn in each 10

2.1 Python (the heart of Django) . . . 10

2.2 HTML . . . 11

2.3 The Django template language (DTL) . . . 11

2.4 CSS . . . 11

2.5 JavaScript . . . 11

2.6 SQL (indirectly, through the ORM) . . . 12

2.7 PowerShell / Batch (Windows scripting) . . . 12

2.8 TOML, YAML and JSON (configuration and data) . . . 12

2.9 Markdown . . . 12

2.10 A learning path to proficiency . . . 12

3. How the web works (the minimum you must know) 13

3.1 Client and server . . . 13

3.2 What a request contains . . . 13

3.3 What a response contains . . . 13

3.4 Stateless: the server forgets you . . . 14

3.5 HTML, CSS, JavaScript: what the browser does with the answer . . . 14

4. Django's shape: projects, apps, settings, MTV 15

4.1 Project vs app . . . 15

4.2 The files startproject creates . . . 15

4.3 settings.py: the control panel . . . 16

4.4 MTV: Model, Template, View . . . 16

4.5 The life of one request, end to end . . . 17

4.6 Middleware: layers around every request . . . 17

4.7 WSGI: how a real web server talks to Django . . . 18

5. URLs and views 19

5.1 The URL table . . . 19

5.2 A view is just a function . . . 19

5.3 The request object . . . 19

5.4 Function-based views (FBV) vs class-based views (CBV) . . . 19

5.5 Decorators on views . . . 21
5.6 Worked example in another app: a library website . . . 21

6. Models, the ORM and migrations 22

6.1 A model is a table . . . 22
6.2 Choices with TextChoices . . . 22
6.3 Relationships . . . 22
6.4 Querying: the ORM . . . 23
6.5 Meta, ordering and indexes . . . 24
6.6 Methods on models . . . 24
6.7 Migrations . . . 24
6.8 Worked example in another app: a shop . . . 24

7. Forms and validation 26

7.1 Why forms exist . . . 26
7.2 The standard view pattern . . . 26
7.3 Field-level and form-level checks . . . 27
7.4 Why the project reuses the form in quick-add . . . 27
7.5 Worked example in another app: a sign-up form . . . 27

8. Templates and static files 28

8.1 The template language in 10 lines . . . 28
8.2 Inheritance: one frame, many pages . . . 28
8.3 Static files . . . 28
8.4 Where Python data meets JavaScript . . . 28
8.5 Worked example in another app: a blog page . . . 28

9. Admin, authentication, sessions, CSRF and messages 29

9.1 The admin site . . . 29
9.2 Authentication and sessions . . . 29
9.3 CSRF protection . . . 29
9.4 The messages framework . . . 29

10. Testing 30

10.1 Why test . . . 30
10.2 Django's test tools . . . 30
10.3 What to test first . . . 30

11. JSON endpoints and talking to Django from JavaScript 31

11.1 The pattern . . . 31

12. Doing work in the background: threads, queues, Redis 32

12.1 The problem . . . 32
12.2 Three ways to solve it . . . 32
12.3 Threads in two minutes . . . 32
12.4 Idempotent jobs . . . 32
12.5 Redis and queues (the upstream library) . . . 32

13. Packaging and running for real 33

13.1 Development server vs production server . . . 33
13.2 Settings per environment . . . 33
13.3 SQLite . . . 33
13.4 PyInstaller . . . 33
13.5 Windows integration used by the app . . . 33

14. Spotting the patterns (the "Django is simple" cheat sheet) 35

15. The map of the repository 37

15.1 Data model at a glance . . . 37
15.2 How an alarm travels . . . 38
15.3 Reading order for teaching . . . 38

16. The testproject/ folder 40

16.1 testproject/manage.py . . . 40
16.2 testproject/testproject/settings.py . . . 40
16.3 testproject/testproject/urls.py . . . 41
16.4 testproject/testproject/views.py . . . 41
16.5 testproject/testproject/wsgi.py and __init__.py . . . 42

17. The desktop/ folder (the Windows program) 43

17.1 desktop/__init__.py . . . 43
17.2 desktop/paths.py . . . 43
17.3 desktop/settings.py . . . 43
17.4 desktop/urls.py . . . 44
17.5 desktop/wsgi.py . . . 44
17.6 desktop/engine.py (the alarm engine) . . . 44
17.7 desktop/main.py . . . 45
17.8 desktop/launcher.py . . . 46
17.9 desktop/build.py . . . 46
17.10 desktop/installer/Install.ps1 and Uninstall.ps1 . . . 46
17.11 manage_desktop.py . . . 47

18. The planner's models: scheduler/models/personal_task.py **48**

18.1 Imports and choice classes . . . 48
18.2 Constants . . . 48
18.3 PersonalTask . . . 48
18.4 SubTask, TaskCompletion . . . 49
18.5 PrayerSchedule and PrayerCompletion . . . 49
18.6 Project, ProjectTask, ProjectSchedule . . . 50
18.7 New in this work: DayPlan, RecurringTask, AlarmSetting . . . 50
18.8 scheduler/models/__init__.py . . . 50
18.9 scheduler/admin/personal_task_admin.py and __init__.py . . . 50

19. The rule modules and forms **52**

19.1 scheduler/planning.py . . . 52
19.2 scheduler/forms.py . . . 52
19.3 scheduler/alarms.py . . . 52
19.4 scheduler/automation.py . . . 53
19.5 Alternative: collapsing the rules into model methods or signals . . . 53

20. The planner's views **55**

20.1 scheduler/views/personal_task_views.py . . . 55
20.2 scheduler/views/mode_views.py (new) . . . 57
20.3 scheduler/views/alarm_views.py (new) . . . 57
20.4 scheduler/views/automation_views.py (new) . . . 58
20.5 scheduler/views/prayer_views.py . . . 58
20.6 scheduler/views/project_views.py . . . 58
20.7 scheduler/views/analytics_views.py . . . 59
20.8 The upstream views (queue_views, worker_views, job_views, queue_job_actions, queue_registry_actions, helpers) . . . 59
20.9 scheduler/views/__init__.py . . . 59

21. URLs and app configuration **60**

21.1 scheduler/urls.py . . . 60
21.2 scheduler/apps.py . . . 60
21.3 scheduler/__init__.py . . . 60
21.4 scheduler/settings.py (the library's own settings loader) . . . 60
21.5 scheduler/decorators.py and the template tag modules . . . 60

22. Templates and static files **62**

22.1 Planner pages . . . 62
22.2 How the pages hand data to JavaScript . . . 62
22.3 The 3D elements (and why they are safe) . . . 63

22.4 Upstream templates (job queue screens) . . . 63
22.5 scheduler/templates/scheduler/ (three files) . . . 63
22.6 Static files . . . 63

23. Migrations, file by file **65**

23.1 Upstream (0001 to 0021) . . . 65
23.2 The planner's migrations (0022 to 0029) . . . 66

24. Tests, file by file **67**

24.1 This work's tests . . . 67
24.2 Upstream tests . . . 67

25. The upstream engine (django-tasks-scheduler), file by file **69**

25.1 scheduler/models/task.py (463 lines): the Task model . . . 69
25.2 scheduler/models/args.py (96 lines) . . . 70
25.3 scheduler/models/ephemeral_models.py (21 lines) and admin/ephemeral_models.py (43 lines) . . . 70
25.4 scheduler/admin/task_admin.py (251 lines) . . . 70
25.5 scheduler/types/ (3 files) . . . 70
25.6 scheduler/redis_models/ (9 files): Redis as a mini database . . . 71
25.7 scheduler/helpers/ (8 files) . . . 71
25.8 scheduler/worker/ (3 files) . . . 72
25.9 scheduler/management/commands/ (6 commands + 2 package markers) . . . 72
25.10 scheduler/tests/ upstream folders . . . 72

26. Repository-level files **73**

27. Glossary **75**

Part I: Theory

1. How to use this guide

This guide teaches Django using one real application: the **Daily Schedule** planner you are building. Real code teaches better than toy examples, because real code has to deal with awkward things: prayer times that nothing may overlap, a limit of two focus tasks a day, alarms that must ring with the window closed.

The guide has two halves.

- **Part I: Theory.** What Django is, how the web works, and every Django idea you need (URLs, views, models, forms, templates, the admin, testing, packaging), each with a small worked example and a diagram. You can read Part I without opening the project.
- **Part II: The project, file by file.** Every file in the repository, in a sensible order, with the reason for each important line and a pointer back to the theory chapter that explains the idea.

Conventions used in the boxes

Four kinds of boxes appear throughout. They are the heart of the guide, because they answer the questions students actually ask.

> **WHY IT IS WRITTEN THIS WAY**
> A green box explains **why** a line is written the way it is. Whenever the code looks like it has an unnecessary step, this box tells you what the step buys.

> **WHAT HAPPENS WITHOUT IT**
> A red box shows **what would go wrong** if you left that line out or did it differently. Seeing the failure is the fastest way to remember the rule.

> **ALTERNATIVE / SIMPLER WAY**
> A blue box shows an **alternative or simpler way**, such as the class-based version of a function view, or a one-line shortcut Django already provides.

> **TIP**
> A grey box is a small practical tip: a command, a habit, a trap to avoid.

How to teach with it

If you are teaching someone else, a good order is: chapter 3 (the web), chapter 4 (Django's shape), then build the smallest possible page (chapter 5), add data (chapter 6), take input (chapter 7), make it pretty (chapter 8). Then open Part II and show how the same five ideas repeat in every file. Almost everything in this project is a variation on those five ideas. Once students see the repetition, Django stops feeling large.

The five ideas that repeat everywhere

Idea	Where you meet it	One-line summary
URL routing	urls.py	A table that maps a web address to a function.
View	views/*.py	A function that takes a request and returns a response.
Model	models/*.py	A Python class that is also a database table.
Form	forms.py	A class that checks what the user typed.
Template	templates/*.html	An HTML file with holes that the view fills in.

Everything else (the admin, migrations, tests, the alarm engine, the .exe) supports these five.

2. Languages used, and what to learn in each

A Django project is never only Python. Even this small planner uses nine languages or file formats. This chapter lists each one, where it appears in the project, and a checklist of what you must be comfortable with to be productive. Use the checklists as a syllabus.

Languages in this project and where each runs



Python runs on the server (inside the .exe). HTML/CSS/JS run in the browser window.

2.1 Python (the heart of Django)

Everything on the server is Python: models, views, forms, the alarm engine, the build script. Django is a Python library, so Python skill is the ceiling on Django skill.

Must know before starting Django

- Variables, numbers, strings, f-strings (`f"Task {title}"`), booleans, `None`.
- Lists, tuples, dictionaries and sets; looping with `for`; list and dict comprehensions.
- Functions: arguments, default values, keyword arguments, `*args` and `**kwargs`, return values.
- Classes: `__init__`, attributes, methods, `self`, **inheritance** (a Django model is a class that inherits from `models.Model`), `@classmethod`, `@property`.
- Modules and imports: `from scheduler.models import PersonalTask`.
- Exceptions: `try / except`, `raise`.
- Decorators: `@require_http_methods(["POST"])` is a decorator; learn what `@something` above a function means.
- The `datetime` module: `date`, `time`, `datetime`, `timedelta`. This project uses it constantly.
- Virtual environments and `pip` / `uv`.

Learn while using Django

- Type hints (`def f(x: int) -> str`), dataclasses (used in `alarms.py`), `threading` (used by the alarm engine), regular expressions (the quick-add parser), context managers (`with`).

Worked mini-example: the same idea in plain Python and in Django

```python
# Plain Python: a class holding data
class Task:
    def __init__(self, title, minutes):
        self.title = title
        self.minutes = minutes

# Django: the SAME class, but it is also a database table
from django.db import models

class Task(models.Model):
    title = models.CharField(max_length=255)
    minutes = models.IntegerField()
```

WHY IT IS WRITTEN THIS WAY

Django models look like normal classes on purpose. You write Python; Django turns it into a table with columns `title` and `minutes`, plus an automatic `id`.

2.2 HTML

HTML is the structure of every page: headings, paragraphs, forms, buttons, tables. Every `.html` file in `scheduler/templates/` is HTML with extra Django syntax mixed in.

Must know: tags and attributes (`<a href="...">`), headings, lists, tables, `<form method="post">` with `<input>`, `<select>`, `<button>`, `<label>`; the difference between block elements (`div`) and inline elements (`span`); the `<head>` and `<body>` split; accessibility basics (labels, `aria-*`, alt text).

2.3 The Django template language (DTL)

A tiny language, not Python, that lives inside HTML files.

```html
{% extends "admin/scheduler/base.html" %}     <!-- reuse a page frame -->
{% block content %}
  <h2>{{ date_display }}</h2>                  <!-- print a variable -->
  {% for t in tasks %}                         <!-- loop -->
    <li>{{ t.title }}</li>
  {% empty %}
    <li>Nothing yet</li>
  {% endfor %}
{% endblock %}
```

Must know: `{{ variable }}`, `{% tag %}`, filters (`{{ x|date:"H:i" }}`), `{% if %}`, `{% for %}`, `{% url 'name' %}`, `{% csrf_token %}`, `{% extends %}` / `{% block %}`, `{% load static %}`, and why templates deliberately cannot run arbitrary Python (keeps design and logic apart).

2.4 CSS

CSS controls colours, spacing, layout and animation. This project uses it for the glass-card look, the dark theme and the 3D tilt effect (`transform: perspective(...) rotateX(...)`).

Must know: selectors (class, id, element, `a:hover > div`), the box model (margin, border, padding), flexbox and grid for layout, units (`px`, `rem`, `%`), colours (hex, rgba), transitions and transforms, media queries (`@media (prefers-reduced-motion: reduce)`), CSS variables.

2.5 JavaScript

JavaScript runs **in the browser**, after Django has already sent the page. In this project it draws the live countdown on the Focus page, sends quick-add text with `fetch()`, ticks task checkboxes without reloading, runs the browser fallback alarm, and drives the Three.js 3D scene.

Must know: variables (`let`, `const`, old-style `var`), functions and arrow functions, the DOM (`document.querySelector`, `addEventListener`, `element.textContent`), events, `fetch()` and JSON, promises

(`.then`), `setInterval` / `setTimeout`, `localStorage`, and the idea that JS cannot see your Python variables, only what Django put into the page.

2.6 SQL (indirectly, through the ORM)

You rarely write SQL in Django, but you must understand what it does, otherwise you cannot reason about speed or correctness.

Must know: tables, rows, columns, primary keys, foreign keys, `SELECT ... WHERE ... ORDER BY`, `JOIN`, `COUNT / SUM`, indexes (why `models.Index` is in `Meta`), transactions, and what SQLite is (a database that lives in one file; used here).

2.7 PowerShell / Batch (Windows scripting)

Used in `desktop/installer/Install.ps1` to copy the program, make shortcuts and register start-with-Windows.
Must know: running commands, variables (`$env:APPDATA`), `if`/`Test-Path`, `Copy-Item`, `Start-Process`.

2.8 TOML, YAML and JSON (configuration and data)

- **TOML** (`pyproject.toml`): project name, dependencies, tool settings.
- **YAML** (`.github/workflows/*.yml`, `mkdocs.yml`, `.pre-commit-config.yaml`): automation and documentation configuration.
- **JSON**: the format every API response in this project uses (`JsonResponse`), and what `fetch()` reads.

2.9 Markdown

`README.md`, `CLAUDE.md` and everything in `docs/` are Markdown: plain text with light formatting symbols. Learn `#` headings, lists, links and code fences.

2.10 A learning path to proficiency

Stage	Goal	What to do
1. Python basics	Read any function in this project	Finish a Python course; write small scripts with dates and dictionaries.
2. Web basics	Explain a request and response	Chapter 3; use browser dev tools (Network tab) on any site.
3. First Django page	URL + view + template	Chapters 4, 5, 8; build a one-page site.
4. Data	Model + migration + admin	Chapter 6; add a Note model and manage it in /admin/.
5. Input	Forms, validation, CSRF	Chapter 7; build create/edit/delete for Notes.
6. JavaScript	Talk to Django from the page	Chapter 12; add a button that ticks a note with fetch().
7. Quality	Tests, migrations in git	Chapter 10.
8. Shipping	Run it for real	Chapter 11 (waitress, PyInstaller).

> **TIP**
> You do not need all nine languages before you begin. Python plus basic HTML is enough to build your first page. Add the others as the project demands them, in the order of the table above.

3. How the web works (the minimum you must know)

Django is easy once you hold one picture in your head: **a website is a function that turns a request into a response.** This chapter builds that picture.

3.1 Client and server

Your browser is the **client**. It asks for things. A **server** is a program waiting for those questions. In this project the server is Django, running inside the `.exe` (or started with `manage.py runserver` while developing). Both live on your own PC, so the address is `127.0.0.1` (also called `localhost`), meaning "this computer".

One HTTP request and its response



The browser asks for a URL. The server runs Python code and answers with text.
Nothing is remembered between requests unless you store it (database, session, cookie).

3.2 What a request contains

Part	Example	Used for
Method	GET, POST	GET = "show me", POST = "change something"
Path	/calendar/2026/11/9/	Which page or action
Query string	?year=2026&month=11	Optional extra values (see calendar_view)
Headers	Cookie, Content-Type	Who you are, what format the body is
Body	title=Gym&date=2026-11-09	Form data, or JSON from fetch()

WHY IT IS WRITTEN THIS WAY
Browsers send **GET** when you click a link or type a URL, and **POST** when a form changes data. Django code branches on `request.method` for exactly this reason: the same URL shows an empty form on GET and processes it on POST (see `create_personal_task`).

WHAT HAPPENS WITHOUT IT
If a delete button used GET, a search engine or a browser pre-fetch could delete your tasks just by visiting links. That is why `delete_personal_task` is limited to POST with `@require_http_methods(["POST"])`.

3.3 What a response contains

A **status code** (200 OK, 302 redirect, 400 bad request, 404 not found, 500 server error), **headers**, and a **body** (usually HTML or JSON). Memorise four codes: **200** worked, **302** go somewhere else (used after a successful form), **400** your request was wrong (used by the JSON endpoints), **404** nothing there.

3.4 Stateless: the server forgets you

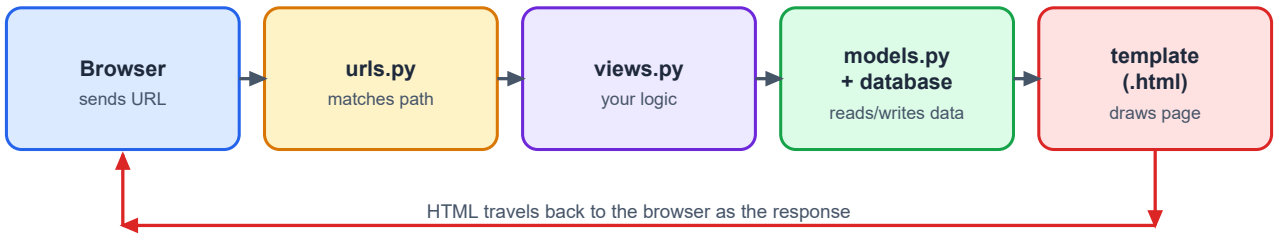
Every request stands alone. The server does not remember the previous one. To remember things Django uses:

- the **database** (tasks, prayers, alarm settings) for data you want to keep,
- **sessions and cookies** for "who is this browser" (used by the admin login and by the `messages` banners),
- **browser storage** (`localStorage`) for small client-only things (the alarm "armed" flag in `alarm.js`).

3.5 HTML, CSS, JavaScript: what the browser does with the answer

The server sends HTML text. The browser draws it, applies CSS, and runs JavaScript. JavaScript can then make *more* requests (`fetch(...)`) without reloading the page. This split is the reason the project has Python files **and** `<script>` blocks.

What Django does with every request



Middleware wraps the whole trip: sessions, CSRF protection, security headers.

Example: /calendar/2026/11/9/ -> urls.py picks task_data_view -> view loads tasks -> template.

A view returns an HttpResponse. render(...) and JsonResponse(...) are shortcuts that build one.

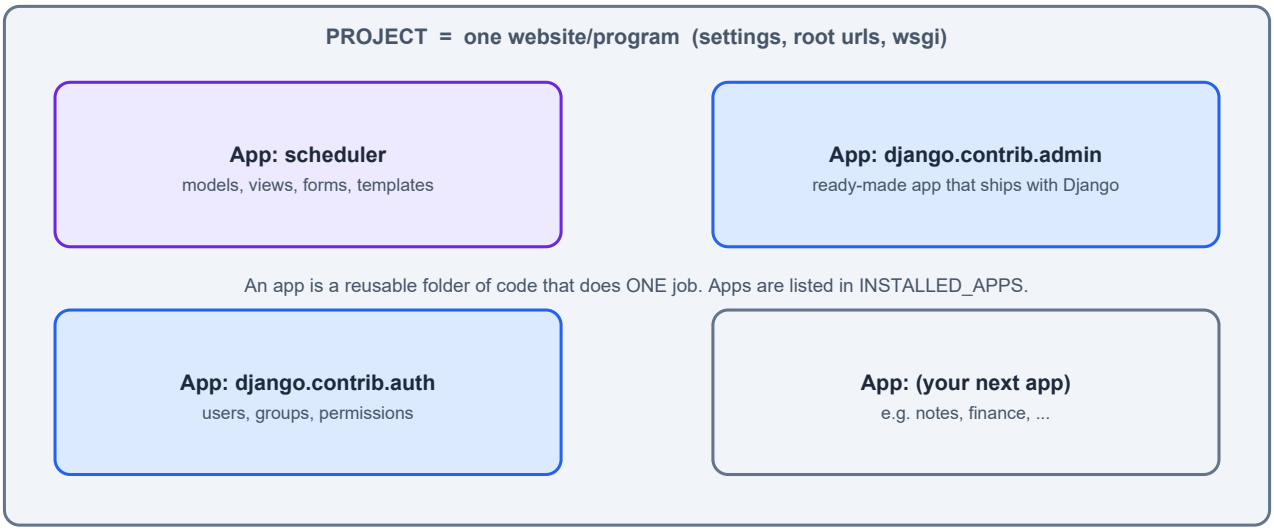
> **TIP**
> Open your browser's developer tools (F12), go to the **Network** tab, and reload any page of the app. Every row is one request/response pair exactly as drawn above. Nothing in Django will feel magical once you have watched this.

4. Django's shape: projects, apps, settings, MTV

4.1 Project vs app

A Django **project** is the whole program: its settings, its root URL table, how it is started. A Django **app** is a reusable package that does one job. A project is made of apps.

Project vs app



In this repository:

- `testproject/` is a small **project** used for development and tests (and `desktop/` is a second project configuration used by the `.exe`).
- `scheduler/` is the **app**. It contains the upstream job-scheduler *and* your planner code.

WHY IT IS WRITTEN THIS WAY

Splitting into apps lets you reuse one app in many projects. The upstream author publishes `scheduler` as a library; your planner rides inside it. The `desktop/` settings reuse the same app with different settings (SQLite in AppData, no real Redis). One app, two projects: that is the benefit.

ALTERNATIVE / SIMPLER WAY

For a first small site you can put everything in one app. Many teams keep one app per business area (users, billing, reports) so each stays small enough to understand.

4.2 The files `startproject` creates

text

```
mysite/
  manage.py            <- the command-line tool for this project
  mysite/
    settings.py        <- every configuration switch
    urls.py            <- the root URL table
    wsgi.py            <- how a web server talks to Django
    asgi.py            <- same, for async servers
```

`manage.py` is a tiny wrapper that points Python at your settings and then runs a Django command. Commands you will use constantly:

Command	What it does
python manage.py runserver	Start a development server that reloads on code changes.
python manage.py makemigrations	Write migration files from model changes.
python manage.py migrate	Apply migrations to the database.
python manage.py createsuperuser	Make an admin login.
python manage.py test	Run the tests.
python manage.py shell	Python prompt with Django loaded (try the ORM here).
python manage.py collectstatic	Gather CSS/JS into one folder for deployment.

4.3 settings.py: the control panel

`settings.py` is just a Python file of UPPERCASE variables. Django reads them by name. The important ones:

```python
INSTALLED_APPS = [                      # which apps are switched on
    "django.contrib.admin",
    "django.contrib.auth",
    "scheduler.apps.SchedulerConfig",    # your app
]
MIDDLEWARE = [ ... ]                    # layers around every request
ROOT_URLCONF = "testproject.urls"       # where the URL table lives
TEMPLATES = [ ... ]                     # how to find and render HTML
DATABASES = {"default": {"ENGINE": "django.db.backends.sqlite3", "NAME": BASE_DIR / "db.sqlite3"}}
STATIC_URL = "/static/"                 # URL prefix for CSS/JS files
DEBUG = True                            # development helper pages; NEVER True in production
SECRET_KEY = "..."                      # signs cookies and sessions
ALLOWED_HOSTS = ["127.0.0.1"]           # which hostnames may reach the app
TIME_ZONE, USE_TZ                       # how dates and times are stored
```

WHY IT IS WRITTEN THIS WAY

One settings file means one place to look. Because it is plain Python it can compute values, read environment variables, or (as `desktop/settings.py` does) create and read a secret key file on first run.

WHAT HAPPENS WITHOUT IT

With `DEBUG = True` in a deployed site, an error shows your source code, settings and database queries to anyone. With a leaked `SECRET_KEY`, an attacker can forge login sessions. That is why `desktop/settings.py` sets `DEBUG = False` and generates a different secret per installation.

4.4 MTV: Model, Template, View

Django's MTV pattern (called MVC elsewhere)



Other frameworks call these: Model = Model, View = Controller, Template = View.

Django's *View* is the traffic cop. It is not the HTML. That naming trips up almost every beginner.

urls.py sits in front: it only chooses WHICH view runs.

- **Model** (`models.py`): what data exists and its rules.
- **View** (`views.py`): the decision-making function for one URL.
- **Template** (`.html`): how the result looks.
- **URLconf** (`urls.py`): a lookup table from URL to view.

Memory trick: **U**rl picks the **V**iew, the View asks the **M**odel, then hands the data to a **T**emplate. U-V-M-T.

4.5 The life of one request, end to end

Worked example: opening the Focus page



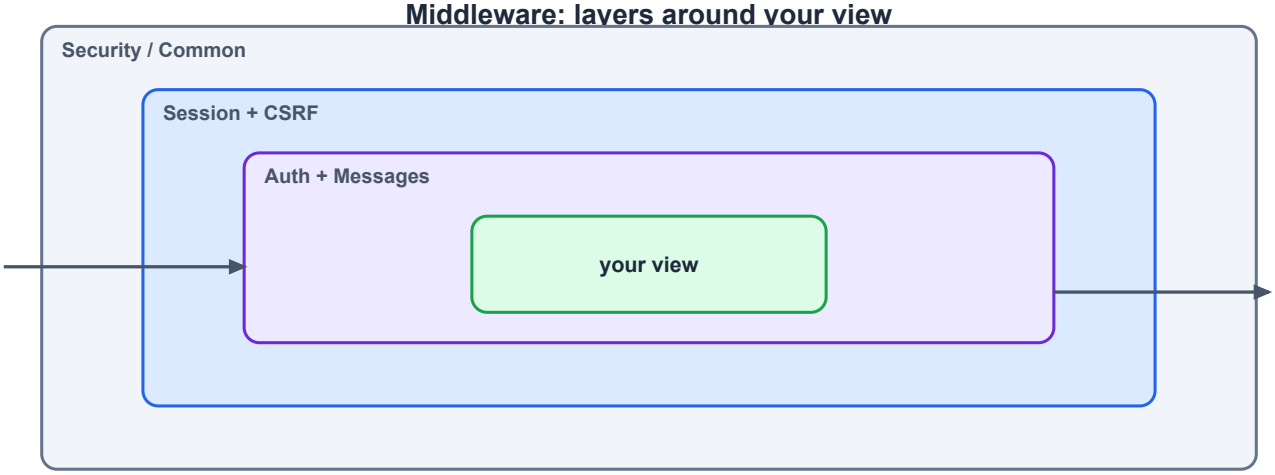
The template receives a context dict (tasks, prayers, slots_left ...) and fills in the HTML.

The browser then runs the page's JavaScript, which draws the live 50/10 countdown.

1. You open `/focus/`.
2. Django's URL table (`scheduler/urls.py`) finds `path("focus/", views.focus_day, name="focus_today")`.
3. `focus_day(request)` runs. It asks the ORM for focus tasks and prayers.
4. It calls `render(request, "admin/scheduler/focus_day.html", context)`.
5. The template is filled in and returned as the response.
6. The browser draws it and runs the page's JavaScript.

4.6 Middleware: layers around every request

Middleware: layers around your view



Requests pass inward through each layer; responses pass outward in reverse order.

Middleware are small classes listed in `MIDDLEWARE`. Each can look at the request on the way in and the response on the way out. Built-ins handle security headers, sessions, **CSRF protection**, login state and the `messages` banners. You rarely write your own, but you must know they exist, because "why is my POST rejected with 403?" is almost always the CSRF middleware doing its job.

4.7 WSGI: how a real web server talks to Django

A web server such as **waitress** (used by the `.exe`) or gunicorn speaks HTTP. Django speaks Python. **WSGI** is the agreed interface between them: a server calls `application(environ, start_response)` once per request. `wsgi.py` exposes that `application` object. `manage.py runserver` includes a development server; for real use you hand `application` to a production server. That is exactly what `desktop/main.py` does with waitress.

5. URLs and views

5.1 The URL table

```python
from django.urls import path
from . import views

urlpatterns = [
    path("calendar/", views.calendar_view, name="calendar_view"),
    path("calendar/<int:year>/<int:month>/<int:day>/", views.task_data_view, name="task_data_view"),
]
```

- `"calendar/"` is the path to match.
- `<int:year>` captures that part of the URL, converts it to an `int` and passes it to the view as the argument `year`. Other converters: `str`, `slug`, `uuid`, `path`.
- `name=` gives the route a nickname so you never hard-code URLs.

WHY IT IS WRITTEN THIS WAY
Naming routes lets templates write `{% url 'task_data_view' year=2026 month=11 day=9 %}` and Python write `reverse("task_data_view", args=[2026, 11, 9])`. If you later change the path to `/day/...`, every link updates itself.

WHAT HAPPENS WITHOUT IT
Hard-coded links like `<a href="/calendar/2026/11/9/">` break silently when a URL changes, and nothing warns you until a user clicks.

5.2 A view is just a function

```python
from django.http import HttpResponse

def hello(request):                 # always receives the request first
    return HttpResponse("Hello")    # always returns a response
```

That is the whole contract. Everything else in this guide is a more convenient way of building the response.

Helper	Use it when	Example in this project
render(request, template, context)	you want an HTML page	task_data_view
redirect("name")	after a successful POST	create_personal_task
JsonResponse({...})	JavaScript is calling	toggle_task_completion
get_object_or_404(Model, id=...)	fetch one row or show a 404 page	edit_personal_task

5.3 The request object

`request.method`, `request.GET["x"]`, `request.POST["x"]`, `request.body` (raw bytes; JSON is read with `json.loads(request.body)`), `request.user`, `request.session`, `request.COOKIES`, `request.FILES`.

5.4 Function-based views (FBV) vs class-based views (CBV)

The same page written both ways. This is the pattern the guide repeats for every view in Part II.

Function-based (what this project uses)

```python
def focus_day(request):
    tasks = PersonalTask.objects.filter(task_type="focus")
    return render(request, "focus_day.html", {"tasks": tasks})
```

Class-based equivalent

```python
from django.views.generic import TemplateView

class FocusDay(TemplateView):
    template_name = "focus_day.html"

    def get_context_data(self, **kwargs):
        ctx = super().get_context_data(**kwargs)
        ctx["tasks"] = PersonalTask.objects.filter(task_type="focus")
        return ctx
```

In `urls.py` a class is registered with `.as_view()`: `path("focus/", FocusDay.as_view())`.

Even shorter, for a plain list page

```python
from django.views.generic import ListView

class TaskList(ListView):
    model = PersonalTask              # Django builds the query: PersonalTask.objects.all()
    template_name = "tasks.html"      # context variable defaults to "object_list"
```

ALTERNATIVE / SIMPLER WAY

Django ships generic views for the common jobs: `ListView` (a list), `DetailView` (one object), `CreateView` / `UpdateView` / `DeleteView` (forms for a model), `TemplateView` (a static page), `RedirectView`. If your page is "list/show/create/edit/delete one model" a CBV can replace 20 lines with 4.

WHY IT IS WRITTEN THIS WAY

This project uses function views because every view here does something specific (prayer checks, focus limits, breaks, automation). A function reads top to bottom, so a student can follow the logic. CBVs shine when you do the standard thing many times; functions win when each view is a one-off.

WHAT HAPPENS WITHOUT IT

Forcing odd logic into a generic view means overriding many hooks (`get_queryset`, `form_valid`, `get_context_data`), which is harder to read than the plain function it replaced.

Create/edit/delete as CBVs (the simplified version of PersonalTask CRUD)

```python
from django.views.generic import CreateView, UpdateView, DeleteView
from django.urls import reverse_lazy

class TaskCreate(CreateView):
    model = PersonalTask
    form_class = PersonalTaskForm             # your validation rules
    template_name = "create_personal_task.html"
    success_url = reverse_lazy("calendar_view")

class TaskUpdate(UpdateView):                 # same, but loads an existing row
    model = PersonalTask
    form_class = PersonalTaskForm
    template_name = "create_personal_task.html"
    success_url = reverse_lazy("calendar_view")

class TaskDelete(DeleteView):
    model = PersonalTask
    success_url = reverse_lazy("calendar_view")
```

Compare with `create_personal_task` and `edit_personal_task`: the function versions add subtasks, auto breaks and a success message, which is why they are longer.

5.5 Decorators on views

A decorator wraps a function to add behaviour.

Decorator	Effect
@require_http_methods(["GET","POST"])	Reject other methods with 405.
@require_POST / @require_GET	Shorthand for one method.
@login_required	Redirect anonymous users to the login page.
@csrf_exempt	Turn OFF CSRF checking for this view (this project removed the risky ones).

> **TIP**
> A very common bug: you add a decorator but forget its import line. Python then says `NameError: name 'require_POST' is not defined`. The import list at the top of every views file exists to prevent exactly that.

5.6 Worked example in another app: a library website

```python
# urls.py
urlpatterns = [
    path("books/", views.book_list, name="book_list"),
    path("books/<int:pk>/", views.book_detail, name="book_detail"),
    path("books/<int:pk>/borrow/", views.borrow, name="borrow"),
]

# views.py
def book_list(request):
    return render(request, "books.html", {"books": Book.objects.order_by("title")})

def book_detail(request, pk):
    book = get_object_or_404(Book, pk=pk)
    return render(request, "book.html", {"book": book})

@require_POST
def borrow(request, pk):
    book = get_object_or_404(Book, pk=pk)
    book.is_available = False
    book.save()
    return redirect("book_detail", pk=pk)
```

Notice the three shapes: **list** (no argument), **detail** (one id), **action** (POST then redirect). The planner's `task_data_view`, `edit_personal_task` and `toggle_task_completion` are the same three shapes. Spotting that is the "trend" the guide wants you to see.

6. Models, the ORM and migrations

6.1 A model is a table

```python
from django.db import models

class Book(models.Model):
    title = models.CharField(max_length=200)
    pages = models.PositiveIntegerField(default=0)
    is_available = models.BooleanField(default=True)
    added = models.DateTimeField(auto_now_add=True)
```

Django creates a table `library_book` with columns `id`, `title`, `pages`, `is_available`, `added`. Every instance is one row.

Field	Stores	Notes
CharField(max_length)	short text	needs max_length
TextField	long text	blank=True, null=True for optional
IntegerField, PositiveIntegerField, FloatField	numbers	
BooleanField	True/False	
DateField, TimeField, DateTimeField	dates and times	auto_now_add sets it on creation
ForeignKey(Other)	link to one row of another model	many-to-one
ManyToManyField	many links both ways	
OneToOneField	exactly one link	

Common options: `default=`, `null=True` (database may store NULL), `blank=True` (forms may leave it empty), `choices=`, `unique=True`, `help_text=`, `verbose_name`.

WHY IT IS WRITTEN THIS WAY

`null` and `blank` are different on purpose. `null` is a database fact; `blank` is a validation fact. A text field you may leave empty should usually be `blank=True` only (store an empty string). That is why `PersonalTask.description` uses both only because it was written that way; a cleaner choice is `blank=True` alone.

6.2 Choices with TextChoices

```python
class TaskMode(models.TextChoices):
    MULTI = "multi", "Multi-Task"
    FOCUS = "focus", "Focus"

task_type = models.CharField(max_length=10, choices=TaskMode.choices, default=TaskMode.MULTI)
```

The first value (`"multi"`) is stored in the database. The second is the label people see. Use `task.get_task_type_display()` to print the label, and compare in code with `task.task_type == TaskMode.FOCUS`.

WHAT HAPPENS WITHOUT IT

Using bare strings everywhere (`if t.task_type == "focsu"`) hides typos until runtime. A named constant (`TaskMode.FOCUS`) makes your editor and tests catch the mistake.

6.3 Relationships

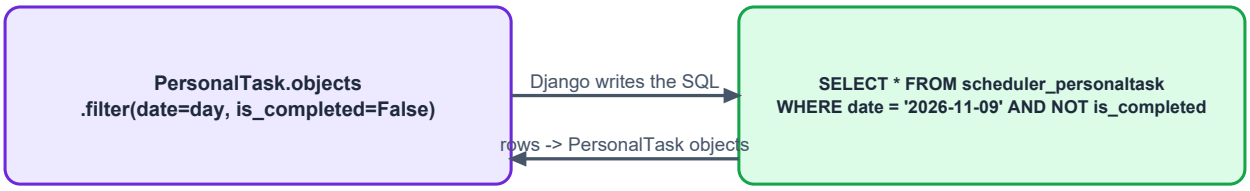
```python
class SubTask(models.Model):
    personal_task = models.ForeignKey(PersonalTask, on_delete=models.CASCADE, related_name="subtasks \
      ")
```

- `ForeignKey` makes each subtask belong to one task.
- `on_delete=CASCADE`: deleting the task deletes its subtasks. Other choices: `SET_NULL` (keep the row, clear the link; used for `recurring_source`), `PROTECT` (refuse to delete).
- `related_name="subtasks"` gives the *reverse* access: `task.subtasks.all()`.

WHAT HAPPENS WITHOUT IT
Without `on_delete` Django refuses to start (it is mandatory), because it cannot guess whether orphaned rows should vanish or block deletion.

6.4 Querying: the ORM

The ORM: Python objects in, SQL out



You never write SQL by hand for normal work. You also never have to worry about SQL injection, because values are passed as parameters, not pasted into the query text.

```python
PersonalTask.objects.all()                                  # every row
PersonalTask.objects.filter(date=day, is_completed=False)   # AND of conditions
PersonalTask.objects.exclude(id=task.id)
PersonalTask.objects.get(id=3)                              # exactly one, else DoesNotExist
PersonalTask.objects.filter(date=day).order_by("start_time")
PersonalTask.objects.filter(date=day).count()
PersonalTask.objects.filter(start_time__lt=end, end_time__gt=start).exists()   # overlap test
PersonalTask.objects.create(title="x", date=day, ...)       # insert
task.save(); task.delete()                                  # update / delete
```

Lookups use double underscores: `__lt`, `__gt`, `__lte`, `__gte`, `__in`, `__contains`, `__icontains`, `__startswith`, and across relationships `subtasks__is_completed=True`.

WHY IT IS WRITTEN THIS WAY
QuerySets are **lazy**. `PersonalTask.objects.filter(...)` runs no SQL until you loop over it, call `.count()` or slice it. You can keep chaining filters and Django builds a single query. The overlap test in `planning.overlaps` and the `.exists()` calls in the views rely on this.

WHAT HAPPENS WITHOUT IT
Looping and calling `.filter()` inside the loop (as `calendar_view` does for each of 42 days) causes the "N+1 queries" problem: 42 separate database trips. It works for a desktop app with tiny data; on a busy site you would use one query with `values()` and `annotate()` instead.

ALTERNATIVE / SIMPLER WAY

For one-trip efficiency across relations use `select_related("fk")` and `prefetch_related("reverse")`. `focus_day` uses `prefetch_related("subtasks")` so that listing every task's subtasks costs two queries total, not one per task.

6.5 Meta, ordering and indexes

```python
class Meta:
    ordering = ["date", "start_time"]
    indexes = [models.Index(fields=["date", "task_type"])]
```

An **index** is a lookup shortcut in the database, like the index in a book. Add one for columns you filter on often. It speeds reads and slightly slows writes.

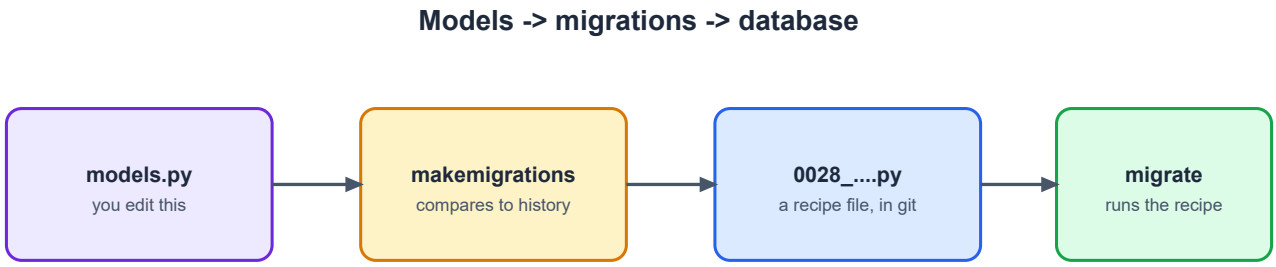
6.6 Methods on models

Put rules about one row on its model. `PersonalTask.get_performance_rating()`, `RecurringTask.runs_on(day)` and `DayPlan.type_for(day)` all live on the model so every view and test uses the same rule.

WHY IT IS WRITTEN THIS WAY

"Fat models, thin views": a rule written once on the model cannot disagree with itself in two views.

6.7 Migrations



Migrations make database changes repeatable: your PC, a test database and the .exe's database all end up with exactly the same tables, in the same order.

Change `models.py`, then:

```text
python manage.py makemigrations scheduler -n recurring_task   # writes scheduler/migrations/0029_rec \
      urring_task.py
python manage.py migrate                                       # applies it
```

A migration is a small Python file saying "create this model" or "add this field". They are numbered and each lists its dependency, forming a chain. Commit them to git.

WHAT HAPPENS WITHOUT IT

Edit a model and skip `makemigrations`, and the app crashes with `no such column`. Edit a migration that others already applied, and databases drift apart. Never edit applied migrations; add a new one.

6.8 Worked example in another app: a shop

```python
class Product(models.Model):
    name = models.CharField(max_length=100)
    price = models.DecimalField(max_digits=8, decimal_places=2)

class Order(models.Model):
    customer = models.ForeignKey("auth.User", on_delete=models.CASCADE)
    created = models.DateTimeField(auto_now_add=True)

class OrderLine(models.Model):
    order = models.ForeignKey(Order, on_delete=models.CASCADE, related_name="lines")
    product = models.ForeignKey(Product, on_delete=models.PROTECT)
    quantity = models.PositiveIntegerField(default=1)

# total of one order
total = sum(l.product.price * l.quantity for l in order.lines.select_related("product"))
```

Compare: `Order` is like `PersonalTask`, `OrderLine` is like `SubTask` (many children, `related_name`, `CASCADE`). `PROTECT` on product mirrors the idea of not letting a delete destroy history.

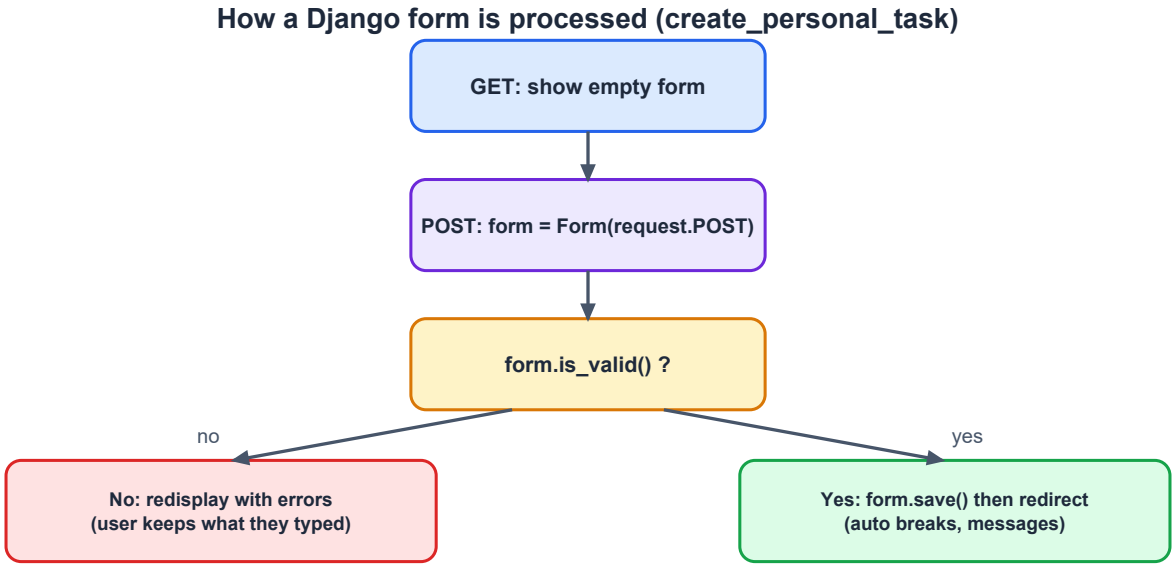
7. Forms and validation

7.1 Why forms exist

User input is untrusted. A Django **form** declares the fields you expect, checks them, converts text to Python types, and gives you clean errors. A **ModelForm** builds the form from a model so you do not repeat yourself.

```python
class PersonalTaskForm(forms.ModelForm):
    class Meta:
        model = PersonalTask
        fields = ["title", "date", "start_time", "end_time", "category", "task_type"]

    def clean(self):                        # rules involving several fields
        data = super().clean()
        if data["start_time"] >= data["end_time"]:
            raise ValidationError("End time must be after start time.")
        return data
```



7.2 The standard view pattern

```python
def create_task(request):
    if request.method == "POST":
        form = PersonalTaskForm(request.POST)
        if form.is_valid():
            form.save()
            return redirect("calendar_view")      # redirect after success
    else:
        form = PersonalTaskForm()                  # empty form on GET
    return render(request, "create.html", {"form": form})
```

Three paths: show empty form, show form with errors (the same template, because `form` now carries the errors), or save and redirect.

> **WHY IT IS WRITTEN THIS WAY**
> **Redirect after POST** prevents the "resubmit form?" problem: if you rendered HTML directly after saving and the user pressed refresh, the browser would POST again and create a duplicate.

WHAT HAPPENS WITHOUT IT
If you skip `is_valid()` and use `request.POST` directly, you accept empty titles, impossible dates and, worse, malicious input.

7.3 Field-level and form-level checks

- `clean_<field>(self)` validates one field.
- `clean(self)` validates combinations, like end after start, or **a task not overlapping a prayer** (the rule this project added).
- Raise `ValidationError("message")`; Django attaches it to the form and your template prints `form.non_field_errors`.

7.4 Why the project reuses the form in quick-add

`automation.quick_add` builds a `PersonalTaskForm` from parsed text instead of creating the row directly. One set of rules (prayer protection, focus limit, no past dates) then protects the form page, the quick-add box and the recurring generator alike.

ALTERNATIVE / SIMPLER WAY
The same validation could go in the model's `clean()` method so even the admin enforces it. Forms are used here because the messages are user-facing and the rule needs request context.

7.5 Worked example in another app: a sign-up form

python

```
class SignUpForm(forms.Form):
    email = forms.EmailField()
    age = forms.IntegerField(min_value=13)
    password = forms.CharField(widget=forms.PasswordInput, min_length=8)

    def clean_email(self):
        email = self.cleaned_data["email"].lower()
        if User.objects.filter(email=email).exists():
            raise forms.ValidationError("That email is already registered.")
        return email
```

A plain `Form` (not tied to a model) is right when the input does not map to one table.

8. Templates and static files

8.1 The template language in 10 lines

```html
{{ task.title }}                        <!-- print -->
{{ task.start_time|time:"H:i" }}        <!-- filter: format -->
{% if task.is_completed %}Done{% else %}Pending{% endif %}
{% for st in task.subtasks.all %}{{ st.title }}{% endfor %}
{% url 'edit_personal_task' task.id %}    <!-- build a URL from a route name -->
{% csrf_token %}                          <!-- hidden security field inside every POST form -->
{% include "partial.html" %}              <!-- paste another file -->
{% load static %}<script src="{% static 'app/alarm.js' %}"></script>
```

WHY IT IS WRITTEN THIS WAY

Templates get **automatic HTML escaping**: `{{ title }}` turns `<script>` into harmless text. That stops "cross-site scripting" attacks without you doing anything.

WHAT HAPPENS WITHOUT IT

Building HTML in Python strings (`"<li>" + title + "</li>"`) is how XSS holes are created. Prefer templates, and only mark text as safe (`|safe`) when you built it yourself.

8.2 Inheritance: one frame, many pages

`base.html` holds the navbar, styles and scripts, with empty `{% block content %}{% endblock %}` holes. Every page starts with `{% extends "admin/scheduler/base.html" %}` and fills only the blocks it needs. Change the navbar once, every page changes.

8.3 Static files

CSS, JavaScript and images live in an app's `static/` folder and are referenced through `{% static 'path' %}`. In development Django serves them automatically. For deployment you run `collectstatic` and serve the collected folder (the `.exe` bundles it and serves `/static/` with `django.views.static.serve`).

8.4 Where Python data meets JavaScript

Django fills the template *before* sending it. JavaScript runs afterwards in the browser and cannot read Python variables, only what was printed into the page: HTML text, `data-*` attributes (like `data-start="{{ t.start_time|time:'H:i' }}"` in the Focus page) or a JSON block. That attribute approach is the simple, safe way to hand server values to scripts.

8.5 Worked example in another app: a blog page

```html
{% extends "base.html" %}
{% block content %}
  <h1>{{ post.title }}</h1>
  <p>By {{ post.author.username }} on {{ post.created|date:"j F Y" }}</p>
  {{ post.body|linebreaks }}
  <h2>Comments ({{ post.comments.count }})</h2>
  {% for c in post.comments.all %}<p>{{ c.text }}</p>{% empty %}<p>No comments yet.</p>{% endfor %}
{% endblock %}
```

9. Admin, authentication, sessions, CSRF and messages

9.1 The admin site

Register a model and Django gives you a full create/read/update/delete website for it.

```python
from django.contrib import admin
@admin.register(PersonalTask)
class PersonalTaskAdmin(admin.ModelAdmin):
    list_display = ("title", "date", "start_time", "category")
    list_filter = ("date", "category")
    search_fields = ("title",)
```

Visit `/admin/` after `createsuperuser`. This is the fastest way to inspect your data while building.

ALTERNATIVE / SIMPLER WAY
You might not need your own CRUD pages at all for an internal tool. This planner has custom pages because the admin cannot show a timeline, a 50/10 timer or a calendar grid.

9.2 Authentication and sessions

`django.contrib.auth` provides `User`, passwords (hashed, never stored plainly), groups and permissions. After login a **session** (a row in the database plus a cookie holding a random id) remembers who you are. `request.user` is then available in every view, and `@login_required` protects a page.

This planner is a **single-user local app**, so its pages are open. If you ever host it online you must add `@login_required` (or `LoginRequiredMixin` for class views) to every view, because otherwise anyone who can reach the address can read and change your tasks.

9.3 CSRF protection

Cross-Site Request Forgery: a malicious page tricks your browser into sending a POST to your site, using your logged-in cookies. Django's defence: every POST must carry a secret token that a foreign page cannot know.

```html
<form method="post">{% csrf_token %} ... </form>
```

For JavaScript requests, read the token and send it in a header:

```javascript
const csrf = document.querySelector('[name=csrfmiddlewaretoken]').value;
fetch(url, {method: 'POST', headers: {'X-CSRFToken': csrf}, body: JSON.stringify(data)});
```

WHAT HAPPENS WITHOUT IT
Marking views `@csrf_exempt` (the original code did, for the AJAX endpoints) turns the protection off. For a purely local app the risk is small, but it is a bad habit to teach, which is why the new endpoints send the token instead.

9.4 The messages framework

`messages.success(request, "Task created")` stores a one-time note in the session; the next template can print it. It exists because after a redirect you cannot pass text in a variable: the page the user lands on is a *new* request.

10. Testing

10.1 Why test

Tests are small programs that run your code and check the answer. They let you change the code later with confidence, and they document the rules ("a third focus task is rejected").

10.2 Django's test tools

```python
from django.test import TestCase

class FocusLimitTest(TestCase):
    def test_third_focus_task_rejected(self):
        make("a", ...); make("b", ...)                    # arrange
        form = PersonalTaskForm(form_data(task_type="focus"))
        self.assertFalse(form.is_valid())                 # act + assert
```

- `TestCase` wraps each test in a transaction and rolls back, so tests never pollute each other or your real database.
- Django creates a **separate empty test database** each run.
- `self.client.get(reverse("home"))` calls your view without a browser; check `res.status_code` and `res.context`.
- Test names must start with `test_`; files with `test_`.

Run: `python manage.py test scheduler.tests.test_modes` (a module) or add `.ClassName.test_name` for one test.

10.3 What to test first

1. Pure functions (parse_quick_add, find_free_slot, events_for_day): no database tricks needed.
2. Rules that protect data (focus limit, prayer overlap).
3. That each page returns 200.
4. Idempotent automation (run twice, same result).

WHY IT IS WRITTEN THIS WAY

The alarm logic was split into `scheduler/alarms.py` (pure calculation) and `desktop/engine.py` (sound and windows) so the calculation can be tested with fake times, while the part that needs speakers cannot be unit tested.

11. JSON endpoints and talking to Django from JavaScript

11.1 The pattern

python

```python
@require_POST
def toggle_task_completion(request, task_id):
    task = get_object_or_404(PersonalTask, id=task_id)
    data = json.loads(request.body)
    task.is_completed = data.get("is_completed", False)
    task.save()
    return JsonResponse({"success": True, "is_completed": task.is_completed})
```

javascript

```javascript
fetch(`/api/tasks/${id}/toggle/`, {
  method: 'POST', headers: {'Content-Type': 'application/json', 'X-CSRFToken': csrf},
  body: JSON.stringify({is_completed: true})
}).then(r => r.json()).then(d => console.log(d.success));
```

Why JSON: it is a text format both Python (`json`) and JavaScript (`JSON.parse`) understand natively.

WHY IT IS WRITTEN THIS WAY

Return **400** for bad input and **200** with `"success": true` for good, so JavaScript can tell what happened from the status code and the body. `quick_add_api` does this and shows the error text on the page.

ALTERNATIVE / SIMPLER WAY

For a large API, use **Django REST Framework** (serializers, viewsets, auth, browsable API). For three endpoints, `JsonResponse` is simpler and has no extra dependency.

12. Doing work in the background: threads, queues, Redis

12.1 The problem

A web request must finish quickly. Things that happen *later* or *repeatedly* (ring an alarm at 14:50, generate tomorrow's recurring tasks) need something that keeps running when no page is open.

12.2 Three ways to solve it

Approach	How it works	Used here
Thread inside the app	A background thread checks the clock every few seconds	desktop/engine.py (the alarms)
Job queue + workers	Jobs are put on a queue in Redis; separate worker processes run them	the upstream django-tasks-scheduler library
Operating-system scheduler	cron / Windows Task Scheduler starts a script at set times	not used (the exe starts itself at login instead)

12.3 Threads in two minutes

A **thread** is a second line of execution inside one program. The alarm engine is a thread that loops: wait 5 seconds, ask "is anything due?", ring if so. Because it is a *daemon* thread it dies when the program quits.

```python
class AlarmEngine(threading.Thread):
    def __init__(self):
        super().__init__(daemon=True)
        self._stop_evt = threading.Event()
    def run(self):
        while not self._stop_evt.wait(5):     # sleeps 5 s, wakes early if stopped
            self.tick(datetime.now())
```

WHAT HAPPENS WITHOUT IT

A plain `while True: time.sleep(5)` can only be stopped by killing the program. `Event.wait(timeout)` is both the sleep and the "please stop" signal.

12.4 Idempotent jobs

A job is **idempotent** if running it twice gives the same result as once. `generate_recurring` checks "does today's copy already exist?" before creating it, so the engine can safely call it every minute and a crash/restart cannot create duplicates. Design every repeated job this way.

12.5 Redis and queues (the upstream library)

Redis is a very fast in-memory store. The upstream scheduler keeps its queues, job states and worker heartbeats there. Your planner does not use it; the desktop build replaces it with an in-memory fake (`FakeRedis`) so no server needs installing.

13. Packaging and running for real

13.1 Development server vs production server

`manage.py runserver` is for development only. For real use serve Django's WSGI `application` with a production server. The desktop app uses **waitress**, a pure-Python server that works on Windows.

13.2 Settings per environment

Keep one settings file per situation. `testproject/settings.py` is for development and tests; `desktop/settings.py` is for the packaged program. They differ in: database location (`%APPDATA%`), `DEBUG`, secret key, cache backend, and the broker.

13.3 SQLite

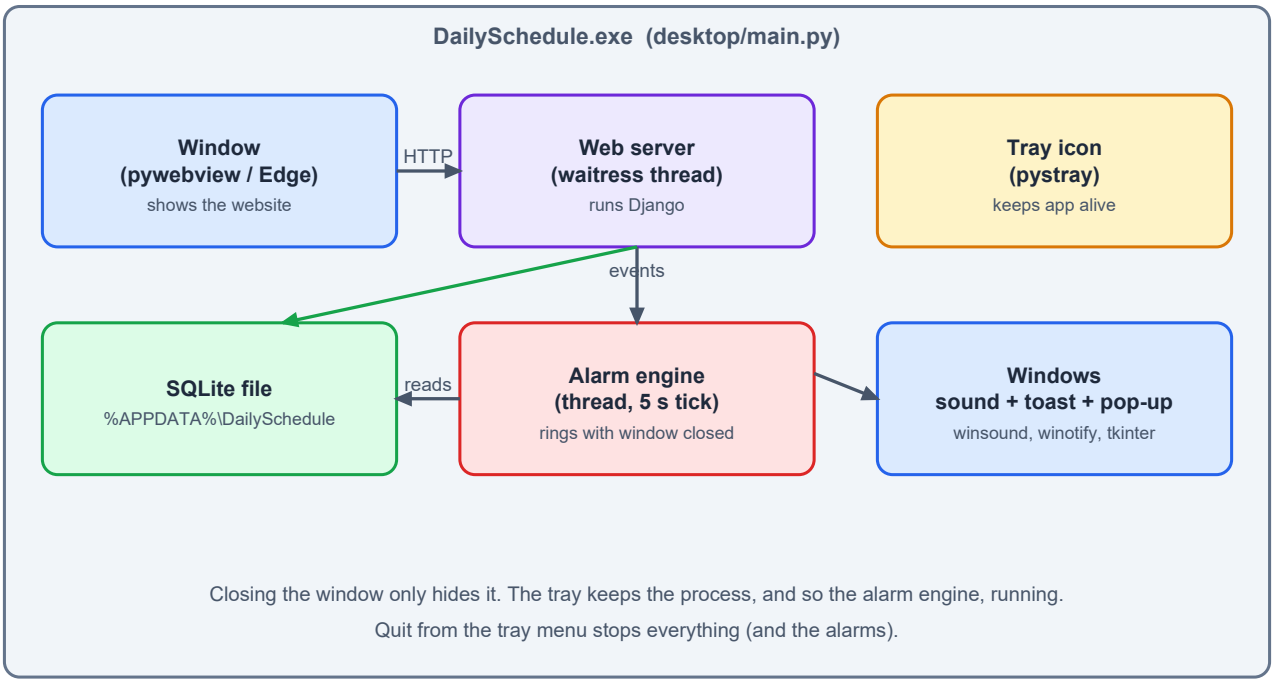
A whole database in one file, no server. It is excellent for a single-user desktop app and for tests. For many simultaneous users switch to PostgreSQL by changing only `DATABASES`; model and view code stays identical (that is the point of the ORM).

13.4 PyInstaller

Bundles Python, your code and every library into a folder with `DailySchedule.exe`. Things to know:

- It cannot always see dynamically-imported modules; the build script lists `--hidden-import` and `--collect-all scheduler` so templates, static files and migrations are included.
- Static files are gathered first with `collectstatic` and bundled.
- Writable data must NOT live next to the exe (it may be read-only); it goes to `%APPDATA%\DailySchedule`.
- `--windowed` hides the console. That also means `print()` shows nowhere, which is why `--selftest` returns an exit code instead of only printing.

DailySchedule.exe - one process, several jobs



13.5 Windows integration used by the app

Feature	Library	Purpose
Native window	pywebview (Edge WebView2)	shows the site like a desktop program
Tray icon	pystray + Pillow	keeps the program running, offers Mute/Quit
Sound	winsound (standard library)	looping alarm tone, volume scaled in the WAV data
Toast notification	winotify	Windows notification centre message
Alarm window	tkinter (standard library)	always-on-top Snooze/Dismiss window
Start with Windows	winreg (standard library)	writes the HKCU\...\Run key
Single instance	ctypes + a named mutex	stops two copies ringing the same alarm

14. Spotting the patterns (the "Django is simple" cheat sheet)

Every file in Part II falls into one of these shapes. If you can name the shape, you can read the file.

Shape	Looks like	Example in this project
List view	filter(...) then render(...)	calendar_view, focus_day
Detail view	get_object_or_404 then render	edit_personal_task (GET)
Create/edit view	form pattern (7.2)	create_personal_task
Action view	POST, change a row, redirect or JSON	delete_personal_task, toggle_*
JSON endpoint	json.loads(request.body), JsonResponse	quick_add_api, set_day_type
Model with rules	fields + Meta + methods	PersonalTask, RecurringTask
Pure helper module	functions with no request in sight	planning.py, alarms.py, automation.py
Template page	extends base, fills block content	home.html, focus_day.html
Migration	Create/Add/Alter operations	0027 to 0029
Test	arrange, act, assert	test_modes.py

Three habits that hold the whole project together:

1. **Thin views, fat helpers.** Views parse the request and choose a response. Rules live in `planning.py`, `automation.py`, `alarms.py` and on the models, where tests can reach them.
2. **One source of truth.** Prayer overlap is checked in one function (`prayer_conflict`) and reused by the form, the break inserter, copy-yesterday, recurring and quick-add.
3. **Make it safe to repeat.** Redirect after POST, idempotent jobs, unique alarm keys.

Part II: The project, file by file

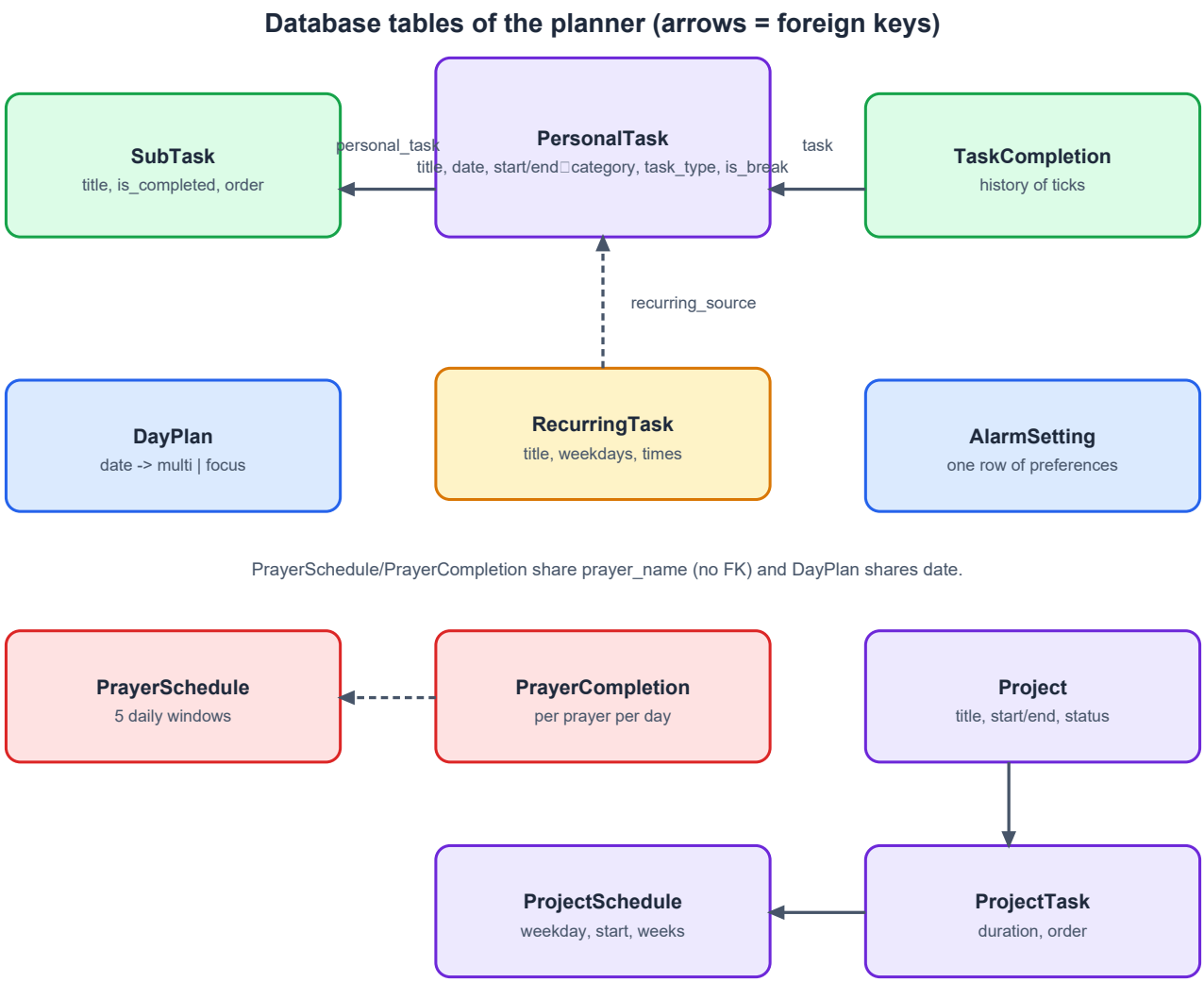
15. The map of the repository

Where things live in the repository

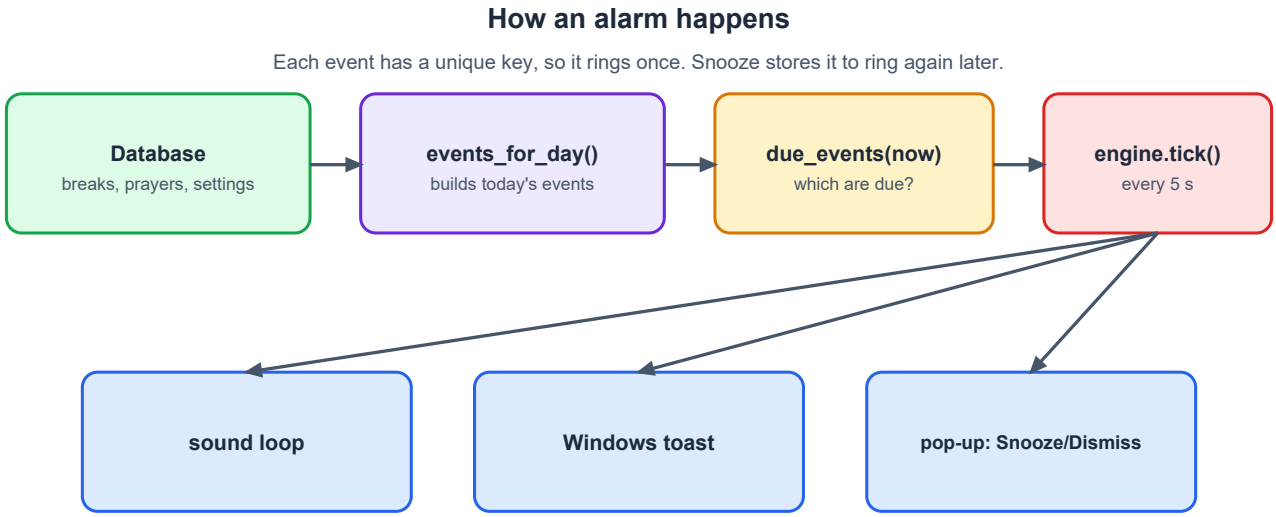
scheduler/	the Django app: models, views, forms, templates, static, tests, migrations
scheduler/worker, redis_models, helpers/	upstream job engine (queues, workers, Redis)
testproject/	minimal Django project used for development and tests
desktop/	the Windows program: settings, alarm engine, tray, build + installer
guide/	this guide: chapters, diagrams, build script
docs/, .github/	upstream documentation and CI workflows
pyproject.toml, uv.lock	dependencies and tool settings
manage_desktop.py	manage.py for the desktop settings

How to read each file entry. Every entry has the same layout: *What it is*, then *Walkthrough* (the important lines and why they exist), then boxes for **why**, **what happens without it**, and **alternatives**, then **Theory link** (the chapter in Part I that teaches the underlying idea).

15.1 Data model at a glance



15.2 How an alarm travels



15.3 Reading order for teaching

1. `testproject/manage.py`, `settings.py`, `urls.py` (how a Django project starts).
2. `scheduler/models/personal_task.py` (the data).
3. `scheduler/urls.py` then `scheduler/views/personal_task_views.py` (a request becomes a page).
4. `scheduler/forms.py` (validation).
5. `scheduler/templates/admin/scheduler/home.html` and `focus_day.html` (the page).
6. `scheduler/planning.py`, `automation.py`, `alarms.py` (rules kept outside views).
7. `desktop/` (turning a website into a program).
8. The tests (`scheduler/tests/test_modes.py` first).
9. Finally the upstream engine (workers, Redis) in chapter 24.

16. The testproject/ folder

16.1 testproject/manage.py

What it is. The command-line entry point for the development project.

```python
#!/usr/bin/env python                       # lets Unix run the file directly; ignored on Windows
import os
import sys

if __name__ == "__main__":                  # only when run as a script, not when imported
    os.environ.setdefault("DJANGO_SETTINGS_MODULE", "testproject.settings")
    from django.core.management import execute_from_command_line
    execute_from_command_line(sys.argv)     # reads "runserver", "migrate", ... from the command lin \
      e
```

WHY IT IS WRITTEN THIS WAY

`DJANGO_SETTINGS_MODULE` is how Django learns which settings file to use; `setdefault` means "use this unless the environment already chose another". That is exactly how the desktop build picks `desktop.settings` instead: it sets the variable first.

WHAT HAPPENS WITHOUT IT

Without the variable Django raises `ImproperlyConfigured: settings are not configured`. Importing Django inside the `if` (not at the top) keeps the variable assignment first, which is required.

ALTERNATIVE / SIMPLER WAY

`manage_desktop.py` at the repository root is the same file with `desktop.settings`. Projects with several environments often use `python manage.py runserver --settings=myproj.settings_dev` instead of several manage files.

Theory link: chapter 4.2.

16.2 testproject/testproject/settings.py

What it is. All configuration for development and for the test suite. Read top to bottom:

```python
BROKER_PORT = os.getenv("BROKER_PORT", "6379")
SCHEDULER_QUEUES = {"default": QueueConfiguration(URL=f"redis://localhost:{BROKER_PORT}/0"), ...}
```

Environment variables (`os.getenv`) let the same file work on different machines. `SCHEDULER_QUEUES` is a setting the **upstream scheduler** reads: a dict of queue names to Redis URLs. Five queues exist so the library's tests can try priorities and databases.

```python
BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
SECRET_KEY = "h0_r$4#4@..."      # fine for a throwaway dev project, NOT for production
DEBUG = True
ALLOWED_HOSTS = ["*"]
```

WHY IT IS WRITTEN THIS WAY

`BASE_DIR` is computed from the file's own location so the project works wherever the folder is copied. `ALLOWED_HOSTS = ["*"]` is acceptable only because this is a local development project.

WHAT HAPPENS WITHOUT IT

A hard-coded absolute path like `C:\Users\...\db.sqlite3` breaks the moment you move the folder or give the project to a colleague.

```python
INSTALLED_APPS = [ "django.contrib.admin", ..., "scheduler.apps.SchedulerConfig" ]
MIDDLEWARE = [ ... CsrfViewMiddleware ... ]
ROOT_URLCONF = "testproject.urls"
CACHES = {"default": {"BACKEND": "django.core.cache.backends.redis.RedisCache", ..., "BROKER": "fake \
      redis"}}
```

`INSTALLED_APPS` ends with the scheduler app; that line is what makes Django find its models, templates and migrations. The cache points to Redis (the upstream tests use FakeRedis).

```python
TEMPLATES = [{"BACKEND": "...DjangoTemplates", "DIRS": [], "APP_DIRS": True, "OPTIONS": {"context_pr \
      ocessors": [...]}}]
```

`APP_DIRS: True` means "look for `templates/` folders inside each installed app". That is why `scheduler/templates/admin/scheduler/home.html` is found as `"admin/scheduler/home.html"`. **Context processors** add variables to every template automatically (`request`, `user`, `messages`).

```python
DATABASES = {"default": {"ENGINE": "django.db.backends.sqlite3", "NAME": os.path.join(BASE_DIR, "db. \
      sqlite3")}}
USE_TZ = False
TIME_ZONE = "UTC"
```

WHY IT IS WRITTEN THIS WAY

`USE_TZ = False` stores plain "naive" local date-times, which suits a personal planner whose times mean "10:00 on my clock". With `USE_TZ = True` every time would be stored in UTC and converted, which is correct for a global site but needlessly confusing here.

WHAT HAPPENS WITHOUT IT

Mixing the two (naive `datetime.now()` while `USE_TZ = True`) triggers warnings and wrong times. The alarm engine uses `datetime.now()` and relies on `USE_TZ = False` for exactly that reason.

Theory link: 4.3, 6.1.

16.3 testproject/testproject/urls.py

```python
urlpatterns = [
    path("admin/", admin.site.urls),
    path("", include("scheduler.urls")),
    path("test-view/", views.my_view),
    path("run-job/", views.run_job),
]
```

`include("scheduler.urls")` mounts the whole app's URL table at the site root. `path("", ...)` means "no prefix".

WHY IT IS WRITTEN THIS WAY

Apps carry their own `urls.py`, so the project file stays tiny and an app can be mounted at `/`, `/planner/` or anywhere.

WHAT HAPPENS WITHOUT IT

If the app had no `urls.py` of its own, every project that used it would have to copy 70 route lines.

16.4 testproject/testproject/views.py

```python
@cache_page(timeout=500)
def my_view(request):
    return HttpResponse("Yeah")

@job("low")
def long_running_func():
    time.sleep(30)

def run_job(request):
    if request.method == "GET":
        long_running_func.delay()
        return HttpResponse("OK - got a GET request")
    return HttpResponse(status=405)
```

Two demo views used by the upstream test suite. `@cache_page` stores the response for 500 seconds. `@job("low")` is the upstream decorator that adds `.delay()`, which puts the function on the Redis queue named `low` for a worker to run. `status=405` is "Method Not Allowed".

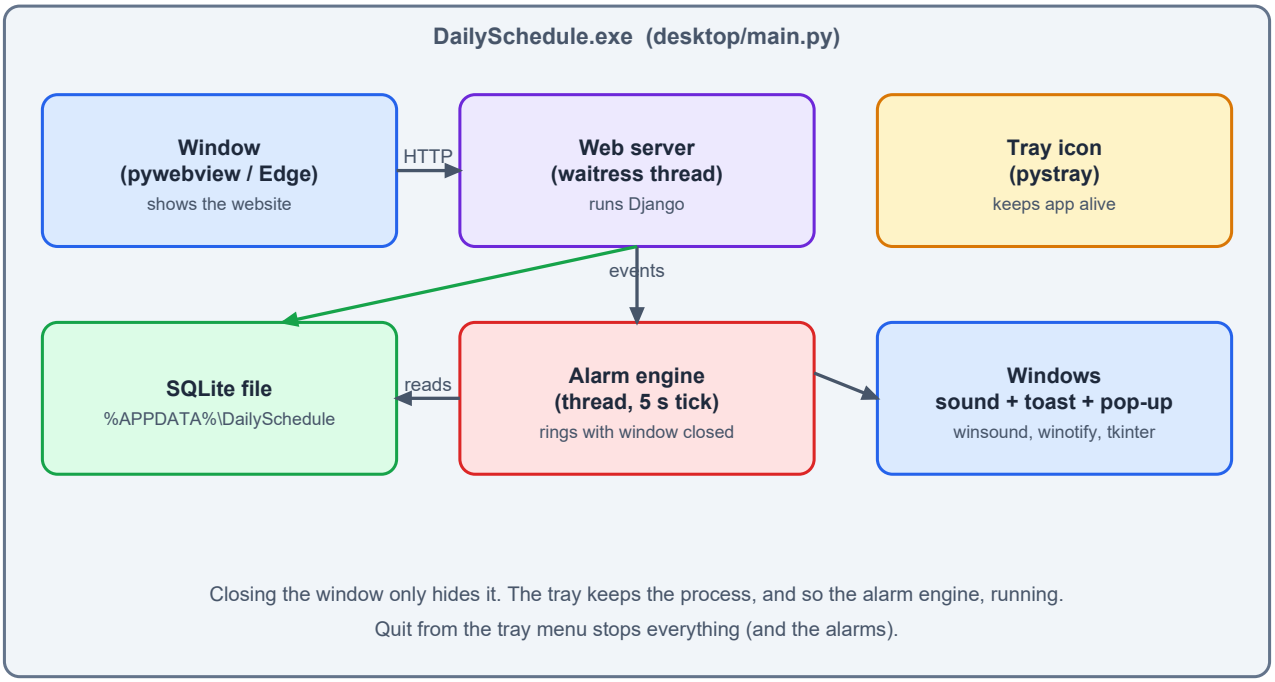
ALTERNATIVE / SIMPLER WAY
The method check could be `@require_GET`, as used elsewhere in this project.

16.5 testproject/testproject/wsgi.py and __init__.py

`wsgi.py` exposes `application`, the WSGI callable (chapter 4.7). `__init__.py` is empty; its existence makes the folder a Python package so `import testproject.settings` works.

17. The desktop/ folder (the Windows program)

DailySchedule.exe - one process, several jobs



17.1 desktop/__init__.py

Empty. Marks `desktop` as a package so `python -m desktop.main` and `from desktop.engine import AlarmEngine` work.

17.2 desktop/paths.py

```python
FROZEN = getattr(sys, "frozen", False)
RESOURCE_DIR = Path(getattr(sys, "_MEIPASS", Path(__file__).resolve().parent.parent))
DATA_DIR = Path(os.environ.get("APPDATA", Path.home())) / APP_NAME
DATA_DIR.mkdir(parents=True, exist_ok=True)
```

- `sys.frozen` is set only inside a PyInstaller program; `getattr(..., False)` is a safe way to read an attribute that may not exist.
- `_MEIPASS` is where PyInstaller unpacks bundled files. When running from source, the repo root is used instead.
- `DATA_DIR` is `%APPDATA%\DailySchedule`: the writable place for the database.

WHY IT IS WRITTEN THIS WAY

Read-only files (templates, static) and writable data (your tasks) must live in different places. Program folders can be read-only and are replaced on update; your tasks must survive updates.

WHAT HAPPENS WITHOUT IT

If the database lived next to the `.exe`, reinstalling or updating would delete your tasks, and a program in `Program Files` could not even write to it.

17.3 desktop/settings.py

Same shape as `testproject/settings.py` with deliberate differences:

Setting	desktop value	Why
DESKTOP_MODE	True	a flag other code can check
DEBUG	False	no source code or settings shown on errors
ALLOWED_HOSTS	127.0.0.1, localhost	only this PC may reach it
SECRET_KEY	random, saved in secret.json	each install has its own key
SCHEDULER_CONFIG	Broker.FAKEREDIS	no Redis server needed
SCHEDULER_QUEUES	{}	the planner uses no queues
CACHES	LocMemCache	in-memory cache inside the process
DATABASES NAME	DATA_DIR / db.sqlite3	data survives updates
STATIC_ROOT	RESOURCE_DIR / staticfiles	bundled by collectstatic

python

```
_secret_file = DATA_DIR / "secret.json"
if not _secret_file.exists():
    _secret_file.write_text(json.dumps({"key": secrets.token_urlsafe(50)}))
SECRET_KEY = json.loads(_secret_file.read_text())["key"]
```

WHY IT IS WRITTEN THIS WAY

`secrets.token_urlsafe` makes a cryptographically strong random key. Writing it once means the same key is reused next launch, so existing sessions stay valid.

WHAT HAPPENS WITHOUT IT

Generating a new key each launch would sign you out every time; shipping one fixed key in the repository would let anyone forge sessions for every copy of the app.

17.4 desktop/urls.py

python

```
urlpatterns = [
    path("admin/", admin.site.urls),
    re_path(r"^static/(?P<path>.*)$", serve, {"document_root": settings.STATIC_ROOT}),
    path("", include("scheduler.urls")),
]
```

With `DEBUG = False` Django no longer serves static files, so this route does it using `django.views.static.serve`. `re_path` takes a regular expression: `(?P<path>.*)` captures the rest of the URL as `path`.

ALTERNATIVE / SIMPLER WAY

The usual production answer is the **WhiteNoise** library (middleware that serves static files efficiently). One extra route avoids a dependency for a single-user program.

17.5 desktop/wsgi.py

Like `testproject/wsgi.py` but defaulting to `desktop.settings`. `main.py` imports `application` from here and hands it to waitress.

17.6 desktop/engine.py (the alarm engine)

What it is. A background thread that rings alarms. Read it in four parts.

Part 1: sound. `build_alarm_wav(volume_pct)` creates a short beep pattern *in memory* as WAV bytes: four tones (880/1100/880/1100 Hz) then a pause. `amp = 32767 * 0.6 * volume/100` scales the loudness. `start_sound` plays it with `winsound.PlaySound(bytes, SND_MEMORY | SND_ASYNC | SND_LOOP)`: from memory, without blocking, repeating. `stop_sound` uses `SND_PURGE`.

WHY IT IS WRITTEN THIS WAY

`winsound` cannot change volume, so the volume is baked into the waveform. Generating the tone in code means no audio file has to ship, and no licence question arises.

WHAT HAPPENS WITHOUT IT

`SND_ASYNC` omitted means the call blocks until the sound ends and the whole engine freezes; `SND_LOOP` omitted means a single beep that the user can miss.

Part 2: toast and "prayed". `toast()` shows a Windows notification with `winotify`; every call is in `try/except` because a notification failure must never stop an alarm. `mark_prayed()` records a prayer using `update_or_create`, the ORM call that updates if the row exists and inserts if not, so pressing the button twice cannot create duplicates (the table also has a unique constraint on `(prayer_name, date)`).

Part 3: the engine class.

```python
class AlarmEngine(threading.Thread):
    def run(self):
        while not self._stop_evt.wait(POLL_SECONDS):
            self.tick(datetime.now())
    def tick(self, now):
        self._automate(now)                          # recurring tasks + missed prayers, at most on \
      ce a minute
        cfg = AlarmSetting.get()
        if self.muted or not cfg.enabled: return []
        candidates = [e for e in due_events(now, cfg) if e.key not in self._fired]
        ...
```

- `_fired` is a set of event keys already rung. This is what makes alarms ring **once**, even though `due_events` keeps returning an event for 90 seconds.
- `_snoozed` holds `(wake_time, event)` pairs; when the time passes they ring again.
- `tick` returns the events it fired, which makes it testable (the tests call it with a fake `now`).
- `self._lock` protects `_snoozed`, because the pop-up window runs on another thread and calls `snooze`.

Part 4: the pop-up window. `_show_window` builds a `tkinter` window: always on top (`-topmost`), centred, with Prayed (prayers only), Snooze and Dismiss buttons, and auto-closing after `ring_seconds`. It runs in its own thread because Tk blocks in `mainloop()`.

WHY IT IS WRITTEN THIS WAY

Heads-up alarms (2 min before a break, 5 before a prayer) only toast and beep once and never open a window, because stealing focus *before* the event would be annoying. Only the real start opens the window.

WHAT HAPPENS WITHOUT IT

Opening a second window while a prayer window is up would bury the prayer; `ring()` explicitly refuses to let a break or task interrupt a ringing prayer.

ALTERNATIVE / SIMPLER WAY

A single-file alternative is `plyer` or `win10toast` for notifications and `playsound` for audio; the standard-library route chosen here has fewer dependencies to bundle.

Theory link: 12 (threads), 13.5 (Windows integration).

17.7 desktop/main.py

```python
os.environ.setdefault("DJANGO_SETTINGS_MODULE", "desktop.settings")   # before Django is imported
```

Function by function:

Function	Job
free_port()	binds port 0 so the OS picks an unused port; avoids "port already in use"
already_running()	named mutex (CreateMutexW); if it exists, a second copy exits
launch_command(), autostart_enabled(), set_autostart()	read/write the HKCU Run registry key
start_server(port)	runs waitress on a daemon thread
wait_for_server(port)	polls /api/alarms/today/ until the site answers, so the window never opens on a blank page
tray_image()	draws the tray icon with Pillow, so no .ico file is needed
main()	the orchestration below

`main()` order of events: parse `--background` / `--selftest`; exit if already running; `django.setup()`; `call_command("migrate")` so a fresh database is created and updated at every start; start the server; wait for it; start the alarm engine; create the window (hidden when `--background`); create the tray; run the GUI loop.

```python
def on_closing():
    if state["quitting"]: return True
    window.hide()          # keep running in the tray so alarms still ring
    return False           # False cancels the close
```

WHY IT IS WRITTEN THIS WAY

Hiding rather than closing is the entire reason alarms work with the "window closed". The process, and so the engine thread, survives. Real quitting is only via the tray menu.

WHAT HAPPENS WITHOUT IT

If closing the window ended the program, your 14:50 prayer alarm would silently never ring after you closed the planner at lunch.

`--selftest` boots everything, checks the API answers, prints and exits with code 0 or 1. It exists because a windowed `.exe` has no console, so a build can be checked automatically.

17.8 desktop/launcher.py

```python
from desktop.main import main
if __name__ == "__main__":
    sys.exit(main())
```

PyInstaller runs a script as `__main__`, where relative imports like `from .paths import ...` would fail. This tiny script imports the package properly and calls `main`.

17.9 desktop/build.py

Runs `collectstatic`, then PyInstaller with these decisions: `--windowed` (no console), `--collect-all scheduler` (include templates, static, migrations, which PyInstaller cannot discover by itself), `--hidden-import` for modules loaded by name (`desktop.settings`, `pystray._win32`, `winotify`), `--add-data staticfiles` for the collected CSS/JS. `package()` then zips the folder with the installer scripts.

17.10 desktop/installer/Install.ps1 and Uninstall.ps1

Install: copies the folder to `%LOCALAPPDATA%\Programs\DailySchedule`, creates Start-Menu and Desktop shortcuts through the `WScript.Shell` COM object, writes the autostart registry value, launches the app. `$ErrorActionPreference = "Stop"` makes any failure abort instead of continuing half-installed. Uninstall removes those and leaves `%APPDATA%\DailySchedule` (your data) untouched.

WHY IT IS WRITTEN THIS WAY

`%LOCALAPPDATA%` and `HKCU` are per-user, so no administrator rights are needed. That keeps the installer simple and safe.

ALTERNATIVE / SIMPLER WAY

A professional installer (Inno Setup, MSIX) gives uninstall entries in Windows Settings and optional code signing; it was not installed on this machine, so scripts were used.

17.11 manage_desktop.py

A copy of `manage.py` with `desktop.settings`. Used by `build.py` to run `collectstatic` against the desktop configuration.

18. The planner's models: scheduler/models/personal_task.py

What it is. Every table of the planner. About 530 lines, but only five shapes repeat. Use the ER diagram in chapter 15.1 as a map.

18.1 Imports and choice classes

```python
from django.db import models
from django.utils.translation import gettext_lazy as _
```

`models` gives fields. `gettext_lazy as _` marks text as translatable: `_("title")` is looked up in a translation catalogue *when displayed* (that is what "lazy" means), not when the file loads. The project is English-only, but the underscore is a Django convention; it costs nothing and lets you add a language later.

```python
class TaskCategory(models.TextChoices):
    NON_NEGOTIABLE = "non_negotiable", _("Non-Negotiable")
    SECONDARY = "secondary", _("Secondary")
    FUN = "fun", _("Fun Activity")
```

Each member: stored value, then label. `TaskCategory.choices` is the list a dropdown needs; `TaskCategory.values` is the list of valid stored strings (used to validate recurring-task input).

`TaskMode` (multi/focus), `PrayerName`, `PrayerCompletionStatus`, `ProjectStatus` follow the identical pattern. **Spot the trend:** every "one of a few options" in the project is a `TextChoices` class.

WHY IT IS WRITTEN THIS WAY
The name `TaskMode` was chosen because the upstream library already defines `TaskType` (once / repeatable / cron). Importing both under one name would silently replace one with the other. When names collide in a large codebase, rename yours.

ALTERNATIVE / SIMPLER WAY
Plain tuples (`CATEGORY_CHOICES = [("a","A"), ...]`) also work, but you lose the named constants.

18.2 Constants

```python
FOCUS_MAX_PER_DAY = 2
POMODORO_WORK_MINUTES = 50
POMODORO_BREAK_MINUTES = 10
```

Rules defined once, imported wherever needed (forms, planning, alarms, views, templates through context). Change 2 to 3 and the whole app follows.

18.3 PersonalTask

Field	Type	Why it exists
title	CharField(255)	what the task is
description	TextField(blank, null)	optional notes
date, start_time, end_time	Date/Time fields	scheduling; times are separate from the date so a recurring or copied task keeps its clock times
category	CharField + TaskCategory	priority label

Field	Type	Why it exists
task_type	CharField + TaskMode, default multi	Multi or Focus
recurring_source	ForeignKey("RecurringTask", SET_NULL)	which template generated it
is_completed	Boolean	ticked off
is_break	Boolean	auto-inserted break rows
parent_task	ForeignKey("self", CASCADE, related_name="breaks")	which task a break belongs to
created_at / updated_at	auto_now_add / auto_now	audit timestamps

WHY IT IS WRITTEN THIS WAY

`ForeignKey("RecurringTask")` uses a **string** because `RecurringTask` is defined lower in the file. Python reads top to bottom, so the class does not exist yet; Django resolves the string later.

WHAT HAPPENS WITHOUT IT

Writing `ForeignKey(RecurringTask, ...)` above its definition raises `NameError`.

`parent_task = ForeignKey("self")` is a **self-reference**: a row pointing to another row of the same table. Deleting a task cascades to its breaks (`CASCADE`), which is why `edit_personal_task` can delete and regenerate breaks safely.

Meta

```python
ordering = ["date", "start_time"]
indexes = [Index(fields=["date"]), Index(fields=["category"]), Index(fields=["date","category"]), In \
      dex(fields=["date","task_type"])]
```

Almost every query filters by `date`, so it is indexed; the combined indexes serve the two-column filters used by the Focus-limit check.

`__str__` returns `"Title (2026-11-09 10:00:00)"` so the admin and shell show readable text instead of `PersonalTask object (7)`.

`get_performance_rating()` counts completed subtasks, converts to a percentage and returns a dict with label and emoji for the UI. It lives on the model because the toggle view, the subtask views and analytics all need it.

ALTERNATIVE / SIMPLER WAY

The long `if/elif` ladder could be a lookup table of `(threshold, label)` pairs and a loop; fewer lines, easier to retune.

18.4 SubTask, TaskCompletion

`SubTask` is the "many children" shape: `ForeignKey` to its task, `order` for sorting, `Meta.ordering = ["order"]`. `TaskCompletion` is an **event log**: every tick or untick adds a row (never updates one), so analytics can see history. This is the "append-only history" shape.

18.5 PrayerSchedule and PrayerCompletion

`PrayerSchedule` has `prayer_name` with `unique=True`: one window per prayer, times editable. `PrayerCompletion` records what happened on a date, with `unique_together = ["prayer_name", "date"]`, so there is at most one row per prayer per day; `status` is prayed / not_yet / missed.

WHY IT IS WRITTEN THIS WAY

Schedule (what *should* happen daily) and completion (what *did* happen on a date) are separate tables. This is the **template vs occurrence** split; you will meet it again as `RecurringTask` vs `PersonalTask`.

WHAT HAPPENS WITHOUT IT
Storing the prayer times inside every day's record would duplicate them 365 times a year, and changing Zhor's time would not change future days.

18.6 Project, ProjectTask, ProjectSchedule

A three-level tree: a `Project` has `ProjectTask`s, each with `ProjectSchedule` rows (weekday + start time + weeks). `unique_together = ["project_task", "day_of_week"]` blocks scheduling the same task twice on a weekday; `generate_schedule` then expands these into real `PersonalTask` rows. `ProjectSchedule.get_end_time()` adds `duration_hours` to the start time with `timedelta`.

18.7 New in this work: DayPlan, RecurringTask, AlarmSetting

DayPlan: `date` (unique) and `day_type`. One row per day *only when you change it*; `DayPlan.type_for(day)` returns the row's type or Multi if none exists. A classmethod keeps that default in one place.

RecurringTask: a template with `weekdays` stored as `"0,2,4"` (0 = Monday). Helper methods: `weekday_set()` parses it, `runs_on(day)` combines active flag and weekday, `weekdays_display()` prints "Mon, Wed, Fri".

ALTERNATIVE / SIMPLER WAY
A comma-separated string is the simplest storage. The cleaner relational approach is a `ManyToManyField` to a Weekday table, or seven boolean columns. The string was chosen for brevity; its cost is that the database cannot filter by weekday.

AlarmSetting: a **singleton**: one row, always `pk=1`, fetched by `AlarmSetting.get()` which uses `get_or_create(pk=1)`. The settings page edits that row.

WHY IT IS WRITTEN THIS WAY
Storing settings in the database (not a file) means the browser page and the desktop engine read the same values with the same code.

WHAT HAPPENS WITHOUT IT
Two sources (a JSON file for the engine, a table for the page) would drift apart.

Theory link: chapter 6 (all of it).

18.8 scheduler/models/__init__.py

Re-exports every model and constant (`from .personal_task import (...)`, plus `__all__`) so other files can write `from scheduler.models import PersonalTask`. Adding a model means adding it in two places here (the import and `__all__`).

WHAT HAPPENS WITHOUT IT
Forgetting the import means `from scheduler.models import RecurringTask` fails, and Django may not detect the model for migrations.

18.9 scheduler/admin/personal_task_admin.py and __init__.py

Each `@admin.register(Model)` class configures the admin: `list_display` (columns), `list_filter` (sidebar), `search_fields`, `readonly_fields` (auto timestamps cannot be edited), `fieldsets` (grouped form sections; `"classes": ("collapse",)` hides a group by default), `date_hierarchy` (date drill-down bar), and `inlines = [SubTaskInline]`, which edits subtasks *inside* the task page (`TabularInline`, `extra = 1` shows one blank row). `__init__.py` imports every admin class so Django registers them.

TIP
Adding `task_type` here took two edits (list and fieldsets). If a field is missing from `fieldsets`, the admin simply does not show it.

19. The rule modules and forms

These four files hold the *rules* of the app, away from requests. They are the best examples of "thin views, fat helpers".

19.1 scheduler/planning.py

```python
def overlaps(a_start, a_end, b_start, b_end):
    return a_start < b_end and a_end > b_start
```

Two time ranges overlap exactly when each starts before the other ends. Because the comparison is strict (`<`, `>`), ranges that merely **touch** (one ends at 13:00, the next starts at 13:00) do not overlap. A test (`test_touching_a_prayer_is_fine`) pins that behaviour.

`prayer_conflict(start, end)` loops over every `PrayerSchedule` and returns the first overlapping one (or `None`). `focus_task_count(day, exclude_pk)` counts focus tasks excluding breaks, and optionally excluding one row: `exclude_pk` is what lets you **edit** an existing focus task without it counting against itself. `focus_slots_left(day)` is `max(0, 2 - count)`.

`auto_insert_breaks(task)` (moved here from the views file) adds a 10-minute break after tasks of 60+ minutes, **shifts it after a prayer** if it would run into one (`break_start = prayer.end_time`), and skips it if another task is in the way.

WHY IT IS WRITTEN THIS WAY
This function moved out of the views file because three places need it now (views, recurring generation, quick-add), and importing from a views module into an automation module would create a circular import.

WHAT HAPPENS WITHOUT IT
With views importing automation and automation importing views, Python raises `ImportError: cannot import name ... (most likely due to a circular import)`.

19.2 scheduler/forms.py

`PersonalTaskForm(ModelForm)`: `Meta.fields` lists what the user may edit (note: `date` and `task_type` are included; `is_completed` and `is_break` are not, so a user cannot forge them by editing the HTML). `Meta.widgets` sets the HTML input type and CSS class per field (`type="time"` gives the browser's time picker).

`clean()` runs after all single-field checks pass, and runs these rules **in order**: (1) start before end; (2) **no overlap with a prayer window**; (3) **at most two focus tasks** that day (excluding itself); (4) not in the past; (5) at most three non-negotiable tasks. The first failure raises `ValidationError` and the message appears at the top of the form.

WHY IT IS WRITTEN THIS WAY
Putting prayer and focus rules in the form's `clean()` means every path that builds this form (the page, the quick-add box) gets them for free.

WHAT HAPPENS WITHOUT IT
A rule enforced only in JavaScript is a suggestion; anyone can bypass it. Enforce in Python, optionally mirror in JS for convenience.

`SubTaskForm`, `PrayerCompletionForm` are plain ModelForms with widgets only.

Theory link: chapter 7.

19.3 scheduler/alarms.py

A small **pure** module: no sound, no windows, no request. `AlarmEvent` is a frozen dataclass (`@dataclass(frozen=True)`: immutable and hashable) with a unique `key`, `kind` (break/prayer/task), `phase` (start or heads_up), title, message and `at`.

- `_events(...)` makes the `start` event and, if `lead > 0`, a `heads_up` event `lead` minutes earlier.
- `pomodoro_events(task, day, lead)` loops `k = 0, 1, 2...`: break starts at `start + 50 + k*60` minutes while that is before the task's end.
- `events_for_day(day, cfg)` returns `[]` if alarms are disabled; otherwise adds prayer events (every day), break events from `is_break` rows, pomodoro events for Focus tasks, and optional task-start events, then sorts by time.
- `due_events(now, cfg, grace_seconds=90)` returns events whose time is within the last 90 seconds.

WHY IT IS WRITTEN THIS WAY
The 90-second grace means an alarm still rings if the computer woke up or the app started a few seconds late, but not an alarm from an hour ago.

WHAT HAPPENS WITHOUT IT
A grace of zero misses any event that falls between two engine ticks; an unlimited grace rings every past alarm of the day the moment you launch the app.

`as_dict()` converts the dataclass to JSON-friendly values (the datetime becomes an ISO string) for the API.

19.4 scheduler/automation.py

Everything the app does for you. Each function is **idempotent**.

Function	Does	Safe-repeat trick
find_free_slot(day, minutes, earliest, latest)	slides a window in 5-minute steps until it avoids every task and prayer	read-only
parse_quick_add(text)	regex parse of "Study maths 4pm 1h #focus"	pure function
quick_add(text, day, default_type)	parse, pick slot, validate through PersonalTaskForm, save, add break	returns (task, error); never raises for user mistakes
generate_recurring(day)	create today's tasks from active templates	skips if `recurring_source` row for that day exists
carry_over_incomplete(from_day, to_day)	re-plan unfinished tasks into free slots	skips a title already on the target day
mark_missed_prayers(now)	flips ended, unprayed prayers to "missed"	only touches `NOT_YET` rows
run_daily_automation(now)	calls the above; the engine runs it every minute	all parts idempotent

Regex notes. `_TIME_RE` accepts `4pm`, `4:30pm`, `at 4pm` or 24-hour `16:30`, but **not** a bare number, so "Read chapter 3" keeps its 3 in the title. `_DUR_RE` accepts `1h`, `1.5 hours`, `45 min`. The time is removed from the text first so "4pm" cannot be mistaken for "4 m" (minutes). `12pm` becomes noon and `12am` midnight via `hour % 12 + (12 if pm else 0)`.

WHY IT IS WRITTEN THIS WAY
`quick_add` returns `(task, error)` instead of raising, because a typo by the user is an expected event, not a program failure. The API turns the error into a 400 response with a message.

19.5 Alternative: collapsing the rules into model methods or signals

You could attach rules to the model (`PersonalTask.clean()` calling `prayer_conflict`) so the admin also enforces them, or use Django **signals** (`post_save`) to insert breaks automatically. Both hide behaviour; this project keeps

the calls explicit in views and helpers so a student can follow them.

20. The planner's views

Reminder of the shapes (chapter 14): list, detail, create/edit, action, JSON endpoint. For each file below, the table says which shape each function is, so you can see the repetition.

20.1 scheduler/views/personal_task_views.py

Function	Shape	URL name	What it does
calendar_view	list	calendar_view	builds a 6x7 month grid with per-day task counts
task_data_view	list + detail	task_data_view	merges tasks and prayers into one sorted timeline
get_tasks_by_date_api	JSON	get_tasks_api	tasks of a date as JSON
create_personal_task	create	create_personal_task	form pattern, subtasks JSON, auto breaks
edit_personal_task	detail + edit	edit_personal_task	same, with `instance=task`
delete_personal_task	action	delete_personal_task	POST only, redirect
toggle_task_completion	JSON action	toggle_task_completion	tick/untick, record a TaskCompletion row
create_subtask / toggle_subtask_completion / delete_subtask / get_task_subtasks	JSON	various	subtask CRUD for JavaScript
get_yesterday_tasks / copy_tasks_from_yesterday	JSON	...yesterday	copy-yesterday feature

calendar_view

```python
today = timezone.now().date()
year = int(request.GET.get("year", today.year))
month = int(request.GET.get("month", today.month))
first_day = datetime(year, month, 1).date()
start_of_week = first_day - timedelta(days=first_day.weekday())
```

`request.GET.get("year", default)` reads `?year=2026` from the URL, falling back to the current year. `weekday()` is 0 for Monday, so subtracting it finds the Monday on or before the 1st; the grid always starts on a Monday. A `while len(calendar_days) < 42` loop adds 6 weeks x 7 days. Then a loop runs three queries per day to count tasks.

WHAT HAPPENS WITHOUT IT

An invalid `?month=13` makes `datetime(year, 13, 1)` raise `ValueError` and the user sees a 500 error page. A careful version would validate or catch it (the other date views do, returning a redirect or a 400).

ALTERNATIVE / SIMPLER WAY

One query for the whole grid, then group in Python:

```python
from django.db.models import Count, Q
rows = (PersonalTask.objects.filter(date__range=[calendar_days[0], calendar_days[-1]])
        .values("date").annotate(total=Count("id"), done=Count("id", filter=Q(is_completed=True))))
task_counts = {r["date"]: r for r in rows}
```

This is 1 query instead of 84. The current code is clearer for a beginner, and fast enough for local data.

task_data_view

Parses `year, month, day` from the URL (`<int:...>` already guaranteed integers, but a date like Feb 31 still fails, hence `try/except ValueError` then `redirect("calendar_view")`). It builds `timeline_items`, a list of dicts mixing prayers and tasks, each with a `type` key (`"prayer"` or `"task"`), then `sort(key=lambda x: x["start_time"])`. Prayer completion is read per prayer with `.first()`. New: each task item carries `task_type` and `task_type_display`, and `day_type` is added to the context for the toggle buttons.

WHY IT IS WRITTEN THIS WAY
The template needs one sorted list, so Python merges two different tables into a common dict shape first. Templates should loop, not decide.

ALTERNATIVE / SIMPLER WAY
A `DayView(TemplateView)` with `get_context_data` is the CBV version. The function version keeps the merging logic in one readable place.

create_personal_task

Walk through it with the form diagram in chapter 7:

1. Parse the date from the URL; invalid, so flash an error and redirect.
2. On POST: `form = PersonalTaskForm(request.POST)`; if valid, `task = form.save(commit=False)` then `task.save()` (`commit=False` returns the unsaved object in case you want to adjust it first).
3. `subtasks_json` is a hidden form field filled by JavaScript; `json.loads` turns it back into a list and each item creates a `SubTask`.
4. `auto_insert_breaks(task)`; `messages.success(...)`; `redirect(...)`.
5. On GET: `initial_data = {"date": date, "task_type": DayPlan.type_for(date)}` pre-fills the date and the day's type.
6. Anything else falls through to `render(...)` with the (possibly error-carrying) form.

WHY IT IS WRITTEN THIS WAY
`initial_data` is what makes "set this day to Focus" change what new tasks default to, without touching the form class.

WHAT HAPPENS WITHOUT IT
If you forget the final `return render(...)` after a failed `is_valid()`, the function returns `None` and Django raises "The view didn't return an HttpResponse object".

ALTERNATIVE / SIMPLER WAY
`CreateView` (chapter 5.4) generates steps 2, 5 and 6; you would still override `form_valid` for subtasks and breaks.

edit_personal_task

Same, plus `get_object_or_404(PersonalTask, id=task_id)` to load the row (a missing id shows a proper 404 page instead of a crash) and `PersonalTaskForm(request.POST, instance=task)` so the form updates that row. After saving it deletes `task.breaks.all()` and regenerates them: a break is derived data, so the simplest correct approach is rebuild.

delete_personal_task, toggle_*, subtask views

Pure **action** shapes. `delete_personal_task`: `@require_http_methods(["POST"])`, delete, message, redirect. `toggle_task_completion`: reads JSON, saves `is_completed`, adds a `TaskCompletion` history row with the performance label, and returns JSON including the subtasks for the page to redraw.

WHY IT IS WRITTEN THIS WAY
These return JSON rather than redirecting because they are called by `fetch()`, which updates part of the page without reloading it.

!without (security note): The original code marked these `@csrf_exempt`. For a local, single-user app that is a low risk; if the app is ever exposed to a network, remove the decorators and send the CSRF header from JavaScript as in chapter 9.3.

copy_tasks_from_yesterday

For each id: load the original; compute `start_time/end_time` (optionally overridden by `time_overrides` from the page); **new:** check the prayer window and the focus limit before creating; create the task (now copying `task_type`); copy subtasks. Errors are collected into a list and returned instead of stopping at the first problem, so one bad task does not block the rest.

20.2 scheduler/views/mode_views.py (new)

Function	Shape	Notes
home	list	counts today's multi and focus tasks (excluding breaks); renders the two big cards
focus_day	list	optional year/month/day URL arguments (default today); loads focus tasks with `prefetch_related("subtasks")`; passes prayers, slots left, 50 and 10
set_day_type	JSON action	validates `day_type in TaskMode.values`, then `update_or_create(date=..., defaults=...)`

```python
def focus_day(request, year=None, month=None, day=None):
```

Two URL patterns point at this one function: `focus/` (no arguments, so the defaults `None` apply) and `focus/<int:year>/<int:month>/<int:day>/`. This is the standard trick to serve "today" and "a given day" with one view.

WHY IT IS WRITTEN THIS WAY

`update_or_create` makes "set the day type" safe to repeat: the first call inserts, later calls update, and `DayPlan.date` being `unique` guarantees one row per day.

ALTERNATIVE / SIMPLER WAY

As a CBV, `focus_day` is a `TemplateView` with `get_context_data`; `set_day_type` stays a function (it is an action, and generic views do not fit actions well).

20.3 scheduler/views/alarm_views.py (new)

- `alarms_today_api` (`@require_GET`): returns `{now, settings, events}` JSON. The browser fallback alarm reads it. `events_for_day` is the same function the desktop engine uses.
- `alarm_settings` (`GET`/`POST`): on POST, boolean checkboxes are set from **presence** in `request.POST` (an unchecked checkbox sends nothing), and integers are clamped to safe ranges with `max(lo, min(hi, value))`; invalid text is skipped with `except ValueError: continue`.

WHY IT IS WRITTEN THIS WAY

Clamping on the server guarantees a stored value of `snooze_minutes=999999` cannot happen even if someone edits the HTML or sends their own POST.

ALTERNATIVE / SIMPLER WAY
A `ModelForm` for `AlarmSetting` would give validation, widgets and error display almost for free. The manual version was kept minimal and shows how forms work underneath.

20.4 scheduler/views/automation_views.py (new)

- `recurring_list` (GET list + POST create): validates title/weekdays/times, creates a `RecurringTask`, then immediately calls `generate_recurring(today)` so a template that applies today appears right away. Warnings from the generator (prayer clash, focus limit) are shown through `messages.warning`.
- `recurring_delete`, `recurring_toggle`: `@require_POST` actions, redirect back.
- `quick_add_api`: `json.loads` the text, call `automation.quick_add`, return 400 with the error text or 200 with the new task's fields.
- `carry_over_api`: calls `carry_over_incomplete(date - 1 day, date)` and returns what moved and what was skipped.

WHY IT IS WRITTEN THIS WAY
Delete is a POST form with a CSRF token, never a link. That is the rule from chapter 3.2.

20.5 scheduler/views/prayer_views.py

Function	Shape
prayer_settings	list
get_prayer_times_api	JSON list (`values(...)` returns dicts, wrapped in `list()` for JSON)
create_prayer / update_prayer	manual form handling (`request.POST.get`, `all([...])` checks, `messages`)
delete_prayer	POST action
toggle_prayer_completion	JSON action with status prayed / missed / not_yet
get_daily_prayer_completions	JSON report with a completion percentage
get_prayer_stats	JSON aggregate: `values("prayer_name").annotate(total=Count("id"), completed=Count("id", filter=Q(is_completed=True)))`

This file shows what Django looks like **without** `ModelForm`: the view reads each `request.POST.get(...)`, checks emptiness by hand and catches broad exceptions. It works, but every field is validated by hand.

ALTERNATIVE / SIMPLER WAY
A `PrayerScheduleForm(ModelForm)` plus `CreateView`/`UpdateView` would remove roughly 60 lines and validate time formats automatically. This is the clearest "simpler Django" opportunity in the project.

20.6 scheduler/views/project_views.py

`projects_list`, `create_project` (accepts JSON or form data by checking `request.content_type`), `project_detail`, JSON actions for tasks and schedules, and `generate_schedule`: for each active `ProjectSchedule`, walk week by week from `project.start_date`, find the next matching weekday with `(schedule.day_of_week - current_date.weekday()) % 7`, compute the end time with `timedelta(hours=...)`, check conflicts, and create `PersonalTask` rows. **Changed in this work:** a candidate that overlaps a prayer window is now reported as a conflict (`reason: "overlaps Zhor"`) instead of being created, because this function creates tasks directly and so bypasses the form rules.

WHAT HAPPENS WITHOUT IT

Before the fix a project schedule at 13:00 on a day where Zhor runs 13:00-13:30 would silently create an overlapping task: exactly the kind of leak a duplicated rule produces. The lesson: any code path that creates tasks must call the same rule function.

20.7 scheduler/views/analytics_views.py

Helpers (`get_date_range`, `calculate_completion_stats`, `get_daily_stats`, the subtask and prayer variants) return plain dicts; `analytics_dashboard` renders the page; `get_chart_data_api`, `get_subtask_chart_data_api`, `get_prayer_chart_data_api` return the Chart.js `labels` and `datasets` JSON.

```python
completed = tasks_query.filter(is_completed=True).count()
total = tasks_query.count()
"percentage": (completed / total * 100) if total > 0 else 0,
```

The `if total > 0 else 0` guard prevents `ZeroDivisionError` on days with no tasks.

TIP

`date__range=[start, end]` is inclusive on both ends and becomes `BETWEEN` in SQL.

20.8 The upstream views (queue_views, worker_views, job_views, queue_job_actions, queue_registry_actions, helpers)

These belong to the upstream library and power the Queues / Workers admin screens. `queue_views.stats` shows queue statistics; `list_registry_jobs` pages through a Redis registry; `queue_workers`, `worker_details`, `workers_list` show workers; `job_detail` and `job_action` act on one job; `queue_job_actions` and `queue_registry_actions` apply bulk actions with a confirmation page. `helpers.py` holds shared pieces: `get_queue` (turns a missing queue into `Http404`), `_find_job`, `_enqueue_multiple_jobs` (re-queues jobs in one Redis **pipeline** for speed) and `_check_next_url`, which defends against **open-redirect** attacks by accepting only same-site "next" URLs (`url_has_allowed_host_and_scheme`). `@never_cache` on stats pages stops the browser from showing stale numbers.

20.9 scheduler/views/__init__.py

A re-export file: imports each view function and lists it in `__all__`, so `urls.py` can write `views.focus_day`. Adding a view requires adding its name here, or `urls.py` fails with `AttributeError: module 'scheduler.views' has no attribute ...`.

21. URLs and app configuration

21.1 scheduler/urls.py

One list built up in blocks with `urlpatterns += [...]` (queues, workers, prayers, calendar/tasks, analytics, projects, then the new alarms, modes, recurring blocks). Patterns to notice:

- **Order matters** only when two patterns could match the same URL; Django uses the first match.
- API routes are grouped under `api/...`, page routes are plain.
- `name=` on every route, so `{% url %}` and `reverse()` work.
- `<int:task_id>` captures an integer; `<str:queue_name>` any text without a slash.

> **ALTERNATIVE / SIMPLER WAY**
> Large apps often split into `urls/` sub-modules per feature, joined with `include()`. A single file is fine under ~100 routes.

21.2 scheduler/apps.py

`SchedulerConfig(AppConfig)` names the app (`name = "scheduler"`), a human label, and a default primary-key type (`default_auto_field = "django.db.models.AutoField"`). `ready()` is an empty hook that runs once at startup; it is where signals are connected if you use them.

21.3 scheduler/__init__.py

Reads the installed package version (`importlib.metadata.version`) and exposes the `@job` decorator (`from scheduler.decorators import job`) so users can write `from scheduler import job`.

21.4 scheduler/settings.py (the library's own settings loader)

Not Django's `settings.py`: this reads `SCHEDULER_QUEUES` and `SCHEDULER_CONFIG` *from* Django settings. `conf_settings()` validates: queues must be a dict; each entry a `QueueConfiguration` or dict; unknown keys in `SCHEDULER_CONFIG` raise `ImproperlyConfigured`. `get_queue_names()` and `get_queue_configuration(name)` serve the rest of the library; an unknown name raises `QueueNotFoundError`. It runs `conf_settings()` at import time so mistakes fail fast at startup, not hours later.

> **WHY IT IS WRITTEN THIS WAY**
> Validating configuration at startup turns a mysterious runtime failure into an immediate, readable error message.

21.5 scheduler/decorators.py and the template tag modules

decorators.py defines `@job(...)` as a **class** that is used like a decorator. `@job("low")` first calls `job.__init__` (it stores the queue and options and resolves a queue *name* into a Queue object). Python then calls the instance on the function: `job.__call__(f)`. That method defines `delay(*args, **kwargs)`, which calls `queue.create_and_enqueue_job(f, args=..., kwargs=...)`, attaches it as `f.delay`, records the function name in `JOB_METHODS_LIST`, and returns the original function unchanged. So `long_running_func()` still runs normally, and `long_running_func.delay()` queues it for a worker.

> **WHY IT IS WRITTEN THIS WAY**
> A class is used because the decorator takes options (`queue`, `timeout`, `at_front`, callbacks). A decorator **with arguments** needs one extra layer; a class with `__init__` (options) plus `__call__` (the wrapping) is a tidy way to write that layer.

WHAT HAPPENS WITHOUT IT

With a plain one-level decorator you could not write `@job("low", timeout=30)`; Python would pass the string to the decorator as the function.

`@wraps(f)` copies the original function's name and docstring onto `delay`, so tracebacks and tools show meaningful names.

templatetags/task_filters.py (11 lines) registers one filter, `get_item`, which returns `dictionary.get(key)` when given a dict. Templates cannot use a variable as a dictionary key (`{{ task_counts.day }}` would look up the literal text "day"), so `{{ task_counts|get_item:day }}` is the workaround (the calendar template uses it).
templatetags/scheduler_tags.py (82 lines) registers filters for the upstream job screens: `show_func_name`, `scheduled_task`, `latest_result`, `job_result`, `job_scheduled_task`, `job_status`, `job_runtime`, `job_scheduled_time`, plus its own `get_item`. A tag library is any module in a `templatetags/` folder that creates `register = template.Library()` and decorates functions with `@register.filter`; templates load it with `{% load scheduler_tags %}`.

ALTERNATIVE / SIMPLER WAY

Prepare the data in the view instead (build `task_counts` as a list of `(day, counts)` pairs) and no filter is needed. Filters are right when the transformation is purely presentational.

22. Templates and static files

All templates live in `scheduler/templates/admin/scheduler/` (plus three duplicates in `scheduler/templates/scheduler/`). The folder name `admin/scheduler/` is a leftover of the upstream library, which overrides admin templates; the planner's pages sit in the same folder.

The base layout idea. `base.html` (890 lines) is a **standalone** page frame, not the Django admin: its own `<style>` block (glass cards, buttons, tables, dark theme), the navbar, the keyboard-shortcuts modal and the script that loads `app/alarm.js`. Almost every planner page begins `{% extends "admin/scheduler/base.html" %}`. The blocks it defines: `title`, `content`, `extra_css`, and one `nav_*` block per navbar link (the page fills its own block with `active` to highlight the link).

WHY IT IS WRITTEN THIS WAY
The `nav_home`, `nav_focus` ... blocks exist so the navbar is written once, in the base, yet each page can mark its own link as active. A page just writes `{% block nav_focus %}active{% endblock %}`.

WHAT HAPPENS WITHOUT IT
Without inheritance you would paste the 900-line frame into every page, and a navbar change would be 25 edits.

22.1 Planner pages

Template	Extends	View	What it shows
base.html	(none)	all	frame, navbar, shortcuts modal, loads alarm.js
home.html	base	home	Three.js hero + two tilting cards (Multi / Focus)
focus_day.html	base	focus_day	task cards, live 50/10 timer, prayer chips, free slots
calendar_view.html	base	calendar_view	month grid; day cells lift in 3D on hover
task_data_view.html	base	task_data_view	day timeline, prayers, tick boxes, day-type toggle, quick add (1,732 lines: mostly inline CSS and JavaScript)
create_personal_task.html	base	create/edit task	the form, category and type help text, subtasks editor
recurring.html	base	recurring_list	create form + table of recurring tasks
alarm_settings.html	base	alarm_settings	alarm preferences form
prayer_settings.html, create_prayer.html, update_prayer.html	base	prayer views	list, add, edit prayer windows
projects_list.html, create_project.html, project_detail.html	base	project views	projects, tasks, schedules, Gantt chart
analytics_dashboard.html	base	analytics_dashboard	Chart.js charts (loaded from a CDN) and stat cards

22.2 How the pages hand data to JavaScript

- `focus_day.html`: `data-start="{{ t.start_time|time:'H:i' }}"` on each card, and an inline JavaScript array built with a `{% for %}` loop for the prayers. The script reads these, never touches Python.

- `task_data_view.html`: `{% csrf_token %}` is placed inside the day-type panel purely so the script can read `[name=csrfmiddlewaretoken]` for its `fetch()` calls.
- `create_personal_task.html`: a hidden `subtasks_json` field is filled by JavaScript before the form submits.

WHAT HAPPENS WITHOUT IT
If the page had no `{% csrf_token %}` anywhere, there would be no token to read, and POSTs from `fetch()` would be rejected with 403.

22.3 The 3D elements (and why they are safe)

- **Tilt cards** (`home.html`, `focus_day.html`): `perspective` on the parent plus `transform-style: preserve-3d` on the card; JavaScript sets `rotateX/rotateY` from the mouse position inside the card and resets on `mouseleave`. Children use `translateZ(40px)` so text floats above the card.
- **Calendar**: `a[href^="/calendar/"]:hover > div { transform: perspective(600px) translateZ(18px) rotateX(8deg); }`. The selector targets the day cell inside each link because those cells are styled inline.
- **Gantt**: `.gantt3d { transform: rotateX(10deg) }` flattens on hover.
- **Three.js hero** (`static/app/hero3d.js`): a torus ring, an icosahedron core, a wireframe shell, and five orbiting dots (one per prayer). The renderer is sized from the canvas, drawing pauses when the tab is hidden (`visibilitychange`), and one static frame is drawn if the user prefers reduced motion.

WHY IT IS WRITTEN THIS WAY
Every effect is wrapped in `@media (prefers-reduced-motion: reduce)` or checks `matchMedia(...)`. Some people feel ill from motion; respecting the operating-system setting is both kind and a basic accessibility requirement.

ALTERNATIVE / SIMPLER WAY
A pure-CSS version of the hero (animated gradient and a rotating `div` cube) needs no library; Three.js was chosen because you asked for real 3D and the library is vendored locally so the app works offline.

22.4 Upstream templates (job queue screens)

Two generations exist side by side. The **old** ones extend `scheduler_base.html`, which extends Django's `admin/base_site.html` (admin look): `stats.html`, `jobs.html`, `job_detail.html`, `queue_workers.html`, `workers_list.html`, `worker_details.html`, `confirm_action.html`, `single_job_action.html`. The **"_new"** versions extend the standalone `base.html` (planner look): `stats_new.html`, `jobs_new.html`, `job_detail_new.html`, `queue_workers_new.html`, `workers_list_new.html`, `worker_details_new.html`, `confirm_action_new.html`, `single_job_action_new.html`. Partials (`jobs-list.partial.html`, `jobs-list-with-tasks.partial.html`, `workers-list.partial.html`) are fragments pulled in with `{% include %}`. `change_form.html` (6 lines) extends Django's `admin/change_form.html` and fills the `after_related_objects` block with `{% include 'admin/scheduler/jobs-list.partial.html' %}`, so a task's page ends with the list of its job executions. `change_list.html` extends `admin/change_list.html`, loads `scheduler_tags`, and overrides `object-tools` with just `{{ block.super }}` (an override that currently changes nothing, a hook left for later).

TIP
Two copies of a template is a maintenance trap: a fix in one is easily missed in the other. Once you are sure the "_new" set is the one in use, delete the old set (check each view's `render(...)` call first).

22.5 scheduler/templates/scheduler/ (three files)

`create_prayer.html`, `prayer_settings.html`, `update_prayer.html` here have the same line counts (116, 116, 129) as the copies in `admin/scheduler/` and extend the same base. The views render the `admin/scheduler/...` names, so these three appear to be unused duplicates and can be removed after confirming.

22.6 Static files

File	Role
scheduler/static/admin/js/select-fields.js	upstream: on the Task admin form it shows only the fields for the chosen task type (Cron, Repeatable or Once) and hides the others, using django.jQuery and the `.tasktype-...` CSS classes
scheduler/static/app/alarm.js	browser fallback alarm: fetches today's events, sets timers, plays a Web Audio tone, shows an overlay with Snooze/Dismiss, asks for notification permission. It exits at once when the user agent contains DailyScheduleDesktop so the exe never double-rings
scheduler/static/app/hero3d.js	the Three.js home scene
scheduler/static/app/vendor/three.min.js	Three.js r128 (600 KB), served locally

WHY IT IS WRITTEN THIS WAY

Browsers refuse to play audio until the user clicks something. The alarm page shows an "Enable alarms" button; its click creates the `AudioContext` and (once) asks for notification permission.

23. Migrations, file by file

A migration file has `dependencies` (the previous migration) and `operations` (what to do). Reading the list below top to bottom is the history of the database.

23.1 Upstream (0001 to 0021)

File	Operations	Story (verified from the file)
0001_initial_squashed_0005_added_result_ttl	3 CreateModel	squashed history; creates CronJob, RepeatableJob and ScheduledJob, the three original job types
0002	3 AlterField	the id column of each job model becomes BigAutoField
0003	2 CreateModel, 5 AlterField	adds the JobArg and JobKwarg argument models; adjusts cron_string and interval_unit (adds "weeks"); ids go back to AutoField
0004	3 AddField	adds at_front (put the job at the front of the queue)
0005	3 AlterField	at_front becomes optional (blank, null)
0006	RunPython	data migration: forward step does nothing; reverse step deletes a job named "Job scheduling jobs"
0007	AddField, 3 AlterField	adds result_ttl; queue field gets help text and a length
0008	6 RemoveField, 2 RenameField	argument models collapse bool/int/datetime value columns into one val column (str_val renamed to val)
0009	4 AlterField	argument type choices and val length
0010	CreateModel	Queue: an unmanaged model (`managed: False`, no table) that only exists so the admin can show a Queues page
0011	CreateModel, AlterModelOptions, 18 AlterField	Worker: another unmanaged admin-only model; many field options updated
0012, 0013, 0014	AlterField	help text and wording for name, queue, created, modified
0015	3 RenameModel	CronJob, RepeatableJob, ScheduledJob become CronTask, RepeatableTask, ScheduledTask
0016	2 RenameModel, 3 AlterModelOptions	JobArg and JobKwarg become TaskArg and TaskKwarg
0017	8 AddField, RemoveField	removes `repeat`; adds failed_runs, last_failed_run, last_successful_run and related bookkeeping
0018	3 AlterField (commented out)	queue-choice experiment; the operations are commented out in the file
0019	CreateModel, 3 AddField	creates the unified Task model and temporary new_task_id links from the three old models

File	Operations	Story (verified from the file)
0020	RemoveField, model removal	drops the temporary links and the old CronTask / RepeatableTask / ScheduledTask models
0021	AddField, RemoveField	job_id replaced by job_name ("current job name on queue")

WHY IT IS WRITTEN THIS WAY
"Unmanaged" models (`managed = False`) have no database table. Django still lets you register them in the admin, which is how the library gets a Queues and a Workers screen backed by Redis instead of a table.

Data migrations (RunPython). 0006 shows the third kind of migration: schema changes alter structure; `RunPython(forwards_func, reverse_func)` runs your own Python to move data. Always give a reverse function so the migration can be undone. Note it uses `apps.get_model(...)` (the historical model), never an import of the real model, because the model may have changed since.

23.2 The planner's migrations (0022 to 0029)

File	Operations	What it created
0022_prayerschedule_personaltask_taskcompletion	3 CreateModel	PrayerSchedule, PersonalTask, TaskCompletion
0023	2 CreateModel, AddField	PrayerCompletion, SubTask, performance_rating
0024	AddField	PrayerCompletion.status
0025	2 AddField	PersonalTask.is_break and parent_task
0026	3 CreateModel, AddIndex	Project, ProjectTask, ProjectSchedule
0027_alarmsetting	CreateModel	AlarmSetting (this work)
0028_task_mode_dayplan	CreateModel, AddField, AddIndex	DayPlan, PersonalTask.task_type, date+task_type index (this work)
0029_recurring_task	CreateModel, AddField	RecurringTask, PersonalTask.recurring_source (this work)

WHY IT IS WRITTEN THIS WAY
`AddField` for `task_type` has `default="multi"`. Existing rows need *some* value when a non-null column is added; the default makes every old task a Multi task, which is exactly the migration rule agreed ("existing data stays Multi").

WHAT HAPPENS WITHOUT IT
Adding a non-null field without a default makes `makemigrations` ask you for one interactively, and `migrate` fail on a table that already has rows.

TIP
Migration numbers are just ordering; two developers creating `0027` at once produce a conflict that `makemigrations --merge` fixes. This is why migrations are committed to git.

24. Tests, file by file

Tests live in `scheduler/tests/`. New tests for this work are marked (new).

24.1 This work's tests

File	Tests	What it proves
test_alarms.py (new)	7	break and prayer events and heads-ups; toggles; completed breaks don't ring; due-window grace; API and settings clamping
test_modes.py (new)	11	third focus task rejected, editing allowed, breaks not counted; prayer overlap rejected in both modes, touching allowed; pomodoro break alarms every 60 min; day-type API; pages render; project generation skips prayer windows
test_automation.py (new)	19	quick-add parsing, free slots avoid prayers, recurring is idempotent, prayer-clash skipped, carry-over, missed prayers, recurring and alarm pages

Pattern in every test: **arrange** (create rows), **act** (call the function or the client), **assert**. Helper functions at the top (`make`, `form_data`) keep each test to a few lines.

```python
def test_touching_a_prayer_is_fine(self):
    self.assertTrue(PersonalTaskForm(form_data(start_time="12:00", end_time="13:00")).is_valid())
```

A good test pins a **boundary**: here, a task ending exactly when the prayer starts must be allowed. If someone later changes `<` to `<=` in `overlaps`, this test fails immediately.

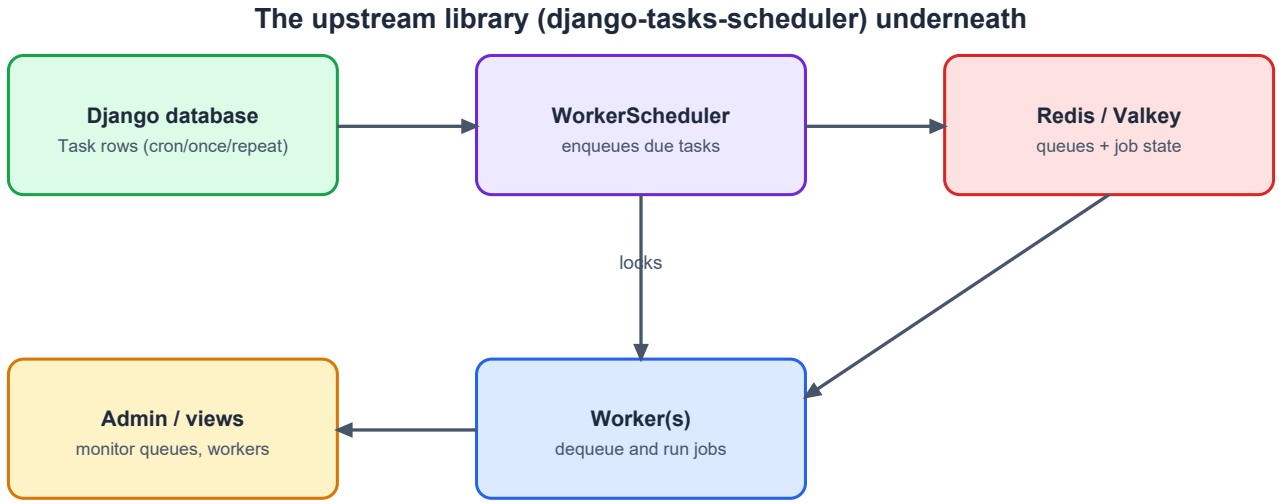
24.2 Upstream tests

File	Tests	Covers
conf.py, jobs.py, testtools.py	helpers	`SchedulerBaseCase` creates queues with FakeRedis; `task_factory` helpers
test_admin.py, test_admin_permissions.py, test_task_admin_display.py	3 / 2 / 1	task admin widgets, read-only permissions, next-run display
test_internals.py	8	internal helpers and settings loading
test_job_arg_models.py	28	TaskArg / TaskKwarg value types
test_job_decorator.py	16	the @job decorator and .delay()
test_queue_getters.py	8	getting queue connections, fail-fast behaviour
test_redis_models.py	9	Redis-backed models, results, admin
test_settings.py	3	SCHEDULER_CONFIG validation
test_mgmt_commands/, test_multiprocess/, test_task_types/, test_views/, test_worker/	folders	export/import commands, multi-process workers, once/cron/repeatable tasks, queue and job views, worker behaviour

> **TIP**
> On Windows the upstream suite fails to load with `ValueError: cannot find context for 'fork'`: some modules ask for the Unix-only multiprocessing start method. This happens with or without this project's changes. The new test modules run fine on Windows.

Run only the new ones with: `cd testproject && set FAKEREDIS=True && uv run python manage.py test scheduler.tests.test_alarms scheduler.tests.test_modes scheduler.tests.test_automation`.

25. The upstream engine (django-tasks-scheduler), file by file



Your planner app piggybacks on this project: it adds PersonalTask etc. next to the Task model.

How honest to be about depth. The planner files (chapters 16 to 24) are explained line by line because they are the ones you wrote and will teach from. The upstream engine is a published library by another author; this chapter explains **what each file is for, how it is structured, and which Django or Python idea it demonstrates**, using the real class and function names from the source. It is the right level to teach "how a larger Django app is organised", and the outlines below were read from the actual files.

The big picture. A `Task` row in the database says "run this function on a schedule". A `WorkerScheduler` process looks for due tasks and puts a **job** on a **queue** stored in Redis. One or more `Worker` processes take jobs off the queue and run them, record results and heartbeats in Redis, and coordinate through locks so two schedulers never enqueue the same job twice.

25.1 scheduler/models/task.py (463 lines): the Task model

Piece	Purpose
`TaskType`	an enum-like choices class: once, repeatable (every N units), cron (a cron expression)
`Task(models.Model)`	one scheduled task: name, callable path, queue, enabled flag, schedule fields per type, timeouts, TTLs, failure bookkeeping (failed runs, last successful run)
`callable_func`, `function_string`	turn the stored dotted path (`"myapp.tasks.send"`) into the real function with an import
`parse_args`, `parse_kwargs`	read the related `TaskArg` / `TaskKwarg` rows into Python lists/dicts
`enqueue_to_run`, `unschedule`, `_schedule_time`	put the job on its queue now or at its next time, or remove it
`get_next_cron_time(...)`	uses the `croniter` library to compute the next time from a cron string
`run_task`, `get_scheduled_task`, `success_callback`, `failure_callback`	the functions a worker actually calls
`get_queue_choices`	builds the queue dropdown from `SCHEDULER_QUEUES`

Why it is a good teaching file: it shows **one model that behaves differently by type** (a "type" column and per-type fields instead of three tables, which the migrations 0019/0020 consolidated), callbacks, and generic relations to arguments.

25.2 scheduler/models/args.py (96 lines)

`BaseTaskArg` is an **abstract base** (shared fields and `clean`/`save`/`value`), with concrete `TaskArg` and `TaskKwarg`. Each argument has a type (str, int, bool, datetime, callable...) and a text value; `value()` converts text back into the right Python type. They link to a task through Django's **generic relations** (`ContentType` + `object_id`), so arguments can belong to any model.

WHY IT IS WRITTEN THIS WAY
Storing every value as text plus a type tag keeps one simple table that can hold any argument kind. The price is a conversion step on read, done in `value()`.

ALTERNATIVE / SIMPLER WAY
Store arguments as one JSON field on the Task (`models.JSONField`). It is simpler to read and write; the arguments inline editor in the admin is what justified separate rows here.

25.3 scheduler/models/ephemeral_models.py (21 lines) and admin/ephemeral_models.py (43 lines)

`Queue` and `Worker` are **placeholder models with no table** (`managed = False`; see migrations 0010 and 0011). They exist only so the admin shows Queues and Workers screens. `ImmutableAdmin` removes add/change permissions, and `QueueAdmin` / `WorkerAdmin` override `changelist_view` to render the Redis-backed pages instead of a table listing.

WHY IT IS WRITTEN THIS WAY
This is the standard trick for showing non-database data in the Django admin: a managed-false model gives the admin something to register and a menu entry, and you replace the page with your own view.

25.4 scheduler/admin/task_admin.py (251 lines)

`TaskAdmin` configures the Task admin: `JobArgInline` and `JobKwargInline` (edit arguments on the same page), `JobMethodsDatalistWidget` (a text input that suggests functions registered with `@job`), `formfield_for_dbfield` to swap that widget in, `task_schedule` and `next_run` (computed columns), admin **actions** `enable_selected`, `disable_selected`, `enqueue_job_now`, and `delete_queryset` / `delete_model` overrides that also remove the job from Redis. Helpers `job_execution_of` and `get_job_executions_for_task` read past executions.

25.5 scheduler/types/ (3 files)

`settings_types.py` defines `SchedulerConfiguration` and `QueueConfiguration` as **dataclasses** (`@dataclass(slots=True, kw_only=True)`; the queue one is also `frozen=True`, so it cannot change after creation) and the `Broker` enum (REDIS, FAKEREDIS, VALKEY). Every tunable has a documented default: `SCHEDULER_INTERVAL = 10` seconds, `DEFAULT_JOB_TIMEOUT = 5 * 60`, `DEFAULT_WORKER_TTL = 10 * 60`, `FAIL_FAST_QUEUE_PROBING = True`, and so on. `broker_types.py` maps each broker to its connection, sentinel and error classes (`BrokerMetaData`). `__init__.py` re-exports the public names.

WHY IT IS WRITTEN THIS WAY
A dataclass gives you typed settings with defaults, a readable repr and typo protection (`scheduler/settings.py` rejects unknown keys). It is the modern replacement for a bag of module-level constants.

WHAT HAPPENS WITHOUT IT
With `slots=True`, assigning a misspelt attribute (`cfg.SCEDULER_INTERVAL = 5`) raises `AttributeError` instead of silently creating a new, unused attribute.

25.6 scheduler/redis_models/ (9 files): Redis as a mini database

File	Contents
base.py (278 lines)	`BaseModel`, `HashModel`, `StreamModel`: a tiny ORM over Redis. A dataclass subclass becomes a Redis hash; `save`, `get`, `get_many`, `delete`, `all`, `count`, `exists`. Helpers `_serialize` / `_deserialize` / `decode_dict` convert between Python and Redis bytes
job.py (337)	`JobModel`, `JobStatus`: one job's state (queued, started, finished, failed, scheduled, stopped, canceled), its function, args, timeouts, callbacks; `prepare_for_execution`, `after_execution`, `expire`, `persist`
result.py (75)	`Result`, `ResultType`: stored output of a run (`create`, `save`, `fetch_latest`)
worker.py (120)	`WorkerModel`, `WorkerStatus`: a worker's registered state, heartbeat and cleanup of dead workers
lock.py (36)	`KvLock` (a lock stored as a Redis key with an expiry), `SchedulerLock`, `QueueLock`
registry/ (folder: `base_registry.py`, `queue_registries.py`)	`JobNamesRegistry` is the base class (plus `DequeueTimeout`); `QueuedJobRegistry`, `ScheduledJobRegistry`, `ActiveJobRegistry`, `FinishedJobRegistry`, `FailedJobRegistry` and `CanceledJobRegistry` each track the names of jobs in one state
`__init__.py`	re-exports the public names (`JobModel`, `Result`, `WorkerModel`, the locks and every registry)

WHY IT IS WRITTEN THIS WAY

Distinct small classes with one job each (a job, a result, a lock) are easier to test than one big module. The locks use **expiring keys**, so if a process crashes its lock disappears on its own and another takes over.

WHAT HAPPENS WITHOUT IT

A lock without an expiry would stay held forever after a crash and the scheduler would never run again.

25.7 scheduler/helpers/ (8 files)

File	Contents
callback.py	`Callback`: wraps a function or dotted path for success/failure/stop hooks, with a timeout; `CallbackSetupError`
timeouts.py	`BaseDeathPenalty`, `UnixSignalDeathPenalty` (uses `SIGALRM`), `TimerDeathPenalty` (a thread timer, used on Windows where `SIGALRM` does not exist), `JobTimeoutException`
sentry_integration.py	optional error reporting to Sentry
utils.py	`current_timestamp`, `utcnow`, `callable_func` (import a function from a dotted path)
queues/getters.py	`get_queue`, `get_queue_connection`, `get_all_workers`, and `_fail_fast_kwargs` (short timeouts so a dead Redis cannot freeze an admin page)
queues/queue_logic.py (432)	the `Queue` class: `create_and_enqueue_job`, `dequeue_any`, `run_sync`, registries, `queue_perform_job`, `get_current_job`, and the exceptions `NoSuchJobError`, `InvalidJobOperation`, `NoSuchRegistryError`

File	Contents
queues/__init__.py	re-exports
__init__.py	empty (package marker)

`_DEATH_PENALTY_CLASS = UnixSignalDeathPenalty if hasattr(signal, "SIGALRM") else TimerDeathPenalty` (in `types/settings_types.py`) is a neat portability line: it picks the timeout mechanism by **feature detection**, not by checking the operating system.

25.8 scheduler/worker/ (3 files)

- `scheduler.py` (164): `WorkerScheduler` acquires the `SchedulerLock`, then loops: `enqueue_scheduled_jobs()` moves due tasks onto queues every `SCHEDULER_INTERVAL` seconds, sends `heartbeat`s and releases the lock on `stop`. `run_scheduler` is the process entry point.
- `worker.py` (895): `Worker` loops `work()`: dequeue a job, `prepare_for_execution`, run it under a timeout, record the result, `handle_job_failure`, update heartbeats, `run_maintenance_tasks`, install signal handlers for clean shutdown. Variants: `SimpleWorker` (runs jobs in-process, easier to debug), `RoundRobinWorker` and `RandomWorker` (change the **dequeue strategy**: how queues are ordered with `reorder_queues`), and `create_worker` builds one from queue names. `StopRequested`, `WorkerNotFound`, `QueueConnectionDiscrepancyError` are its custom exceptions.
- `__init__.py` exposes the public pieces.

> **ALTERNATIVE / SIMPLER WAY**
> For most Django projects Celery (heavier, many features) or Django-Q / Huey / django-tasks (lighter) fill the same role. This library keeps the schedule **in the Django database and admin**, which is its selling point.

25.9 scheduler/management/commands/ (6 commands + 2 package markers)

A management command is a class `Command(BaseCommand)` with `add_arguments(parser)` (declare options) and `handle(*args, **options)` (do the work), run as `python manage.py <filename>`.

Command	Verified options	Does
scheduler_worker	--name, --burst, --without-scheduler, --worker-ttl, --max-jobs, --max-idle-time, --fork-job-execution, --pid, --sentry-dsn, --sentry-debug, --sentry-ca-certs	start a worker (and its scheduler, unless told not to) on the named queues; `--burst` exits when the queues are empty
scheduler_stats	--interval, --json, --yaml	print queue statistics as a text dashboard; `--interval` repeats it
export	--filename, --output, --enabled, and a format of json or yaml (default json)	export scheduled tasks
import	--filename, --format, --reset, --update	import tasks from a file; `--reset` clears first, `--update` updates matches
run_job	--queue, --timeout, --result-ttl, plus positional args	enqueue a named function immediately
delete_failed_executions	--queue, --func, --dry-run	purge failed job records, optionally only one function or only a preview
__init__.py (two of them)	package markers for `management` and `commands`	

25.10 scheduler/tests/ upstream folders

Described in chapter 24.2; the sub-folders (`test_worker`, `test_views`, `test_task_types`, `test_mgmt_commands`, `test_multiprocess`) mirror the packages they test.

26. Repository-level files

File	What it is	Why it exists
pyproject.toml	project metadata, dependencies, optional extras (`yaml`, `valkey`, and the new `desktop`), ruff and mypy settings	single modern place for Python packaging and tool config; `[dependency-groups]` lists dev tools
uv.lock	exact versions of every dependency	reproducible installs
.gitignore	files git must skip (venv, caches, build output, now `staticfiles/`, `*.spec`)	keeps generated files out of the history
.pre-commit-config.yaml	hooks run before each commit: check-json/xml/yaml, end-of-file and whitespace fixers, codespell, ruff (lint and format), pyproject-fmt, and more	consistent code style without arguing about it
CLAUDE.md	instructions for the Claude Code assistant (commands, architecture summary)	so an AI helper follows the project's conventions
README.md, SECURITY.md, CODE_OF_CONDUCT.md, LICENSE	project description, how to report vulnerabilities, behaviour rules, MIT licence	standard open-source files
mkdocs.yml, .readthedocs.yaml	documentation site configuration	builds `docs/` into a website
docs/*.md (index, installation, configuration, usage, commands, drt-model, migrate_to_v3, changelog)	upstream documentation; `drt-model.md` describes worker flows; `migrate_to_v3.md` upgrade notes	user manual of the library
docs/media/*.jpg (9 images)	screenshots of the admin pages used in the documentation	illustrations
.github/workflows/test.yml	"Django CI": on push/PR runs tests across a matrix of Python version, Django version and broker	automatic testing
.github/workflows/publish.yml	"Release": builds and publishes the package on a release	automatic release
.github/workflows/publish-documentation.yml	builds and publishes the docs on a release or manually	docs site
.github/workflows/test-workflow.yml	a manually triggered lint (ruff) workflow	on-demand check
.github/actions/test-coverage/action.yml	a reusable action that runs tests with coverage and comments the result on a pull request	coverage reporting
.github/zizmor.yml, dependabot.yml, release-drafter.yml	workflow security linting rules, automatic dependency update PRs, release-note drafting	repository hygiene
.github/CONTRIBUTING.md, FUNDING.yml, ISSUE_TEMPLATE/*.md	contributor guide, sponsorship link, bug and feature request templates	community process
.claude/settings.json and testproject/.claude/settings.json	Claude Code permission settings	assistant configuration
manage_desktop.py	`manage.py` for the desktop settings	collectstatic for the build
guide/ (build_guide.py, diagrams.py, chapters/*.txt)	this guide	documentation as code

WHY IT IS WRITTEN THIS WAY

Notice that every file in the repository root has exactly one job. When you start your own project, create these early: a `.gitignore`, a `pyproject.toml` (or `requirements.txt`), a README and a lock file. They save more time than any feature.

27. Glossary

Term	Meaning
Request / Response	what the browser sends and what the server returns
View	a Python function (or class) turning a request into a response
URLconf	the table of URL patterns to views
Model	a Python class that is a database table
Migration	a recorded database change
ORM	Object-Relational Mapper: Python objects instead of SQL
QuerySet	a lazy, chainable database query
Template	an HTML file with Django placeholders
Context	the dictionary a view hands to a template
Form / ModelForm	a validated input description, optionally generated from a model
Middleware	code that wraps every request and response
CSRF	forged cross-site requests; blocked with tokens
Session	server-side memory about one browser, found through a cookie
Static files	CSS, JavaScript and images
WSGI	the interface between a web server and Django
Idempotent	safe to run twice with the same result
Daemon thread	a background thread that dies with the program
Singleton	a table that holds exactly one row (AlarmSetting)
Template vs occurrence	a reusable plan versus one dated instance (RecurringTask vs PersonalTask)
Unmanaged model	a model with no table, used for admin pages
Fixture / test database	throwaway data and database used during tests
PyInstaller	tool that bundles a Python program into an .exe
Tray icon	the small icon near the Windows clock that keeps the program alive